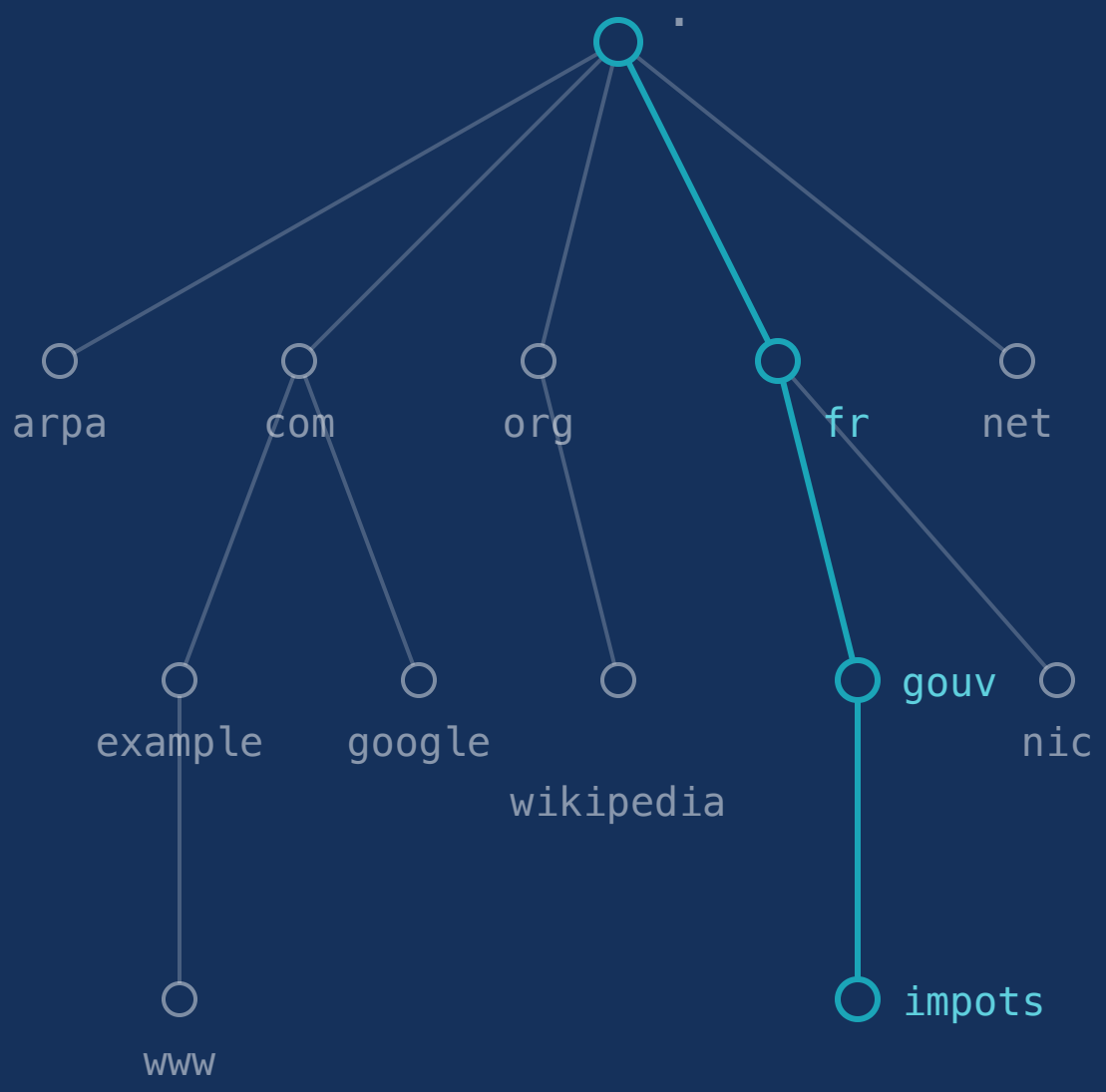
IUT BLAGNAC

Domain Name System Fundamentals

Julien MANTEAU

2025



Objectives

- Describe the principles of DNS and how its data are organized (namespace, zones, records)
- Trace a full DNS resolution with dig and name the servers involved
- Explain how caching and TTL affect the answers you see
- Describe advanced DNS concepts (reverse lookup, split horizon, round robin, zone transfer)
- Explain the role of DNS in the kill chain (the stages of a cyberattack)
- Run a DNS C2 channel in a lab, detect it in the logs and block it with RPZ
- Match DNS attacks to their defenses (DNS firewall, DoT, DoH, DoQ, DNSSEC)

Teacher Presentation: Julien MANTEAU

- Network & Telecom Engineer (ENSEEIHT)



- Lecturer at Béziers then Blagnac IUT in “Réseaux Télécoms” from 2010 to 2020
- Leader on Airbus CyberDiploma cursus setup and teacher/lecturer since 2021
- Hobbies:

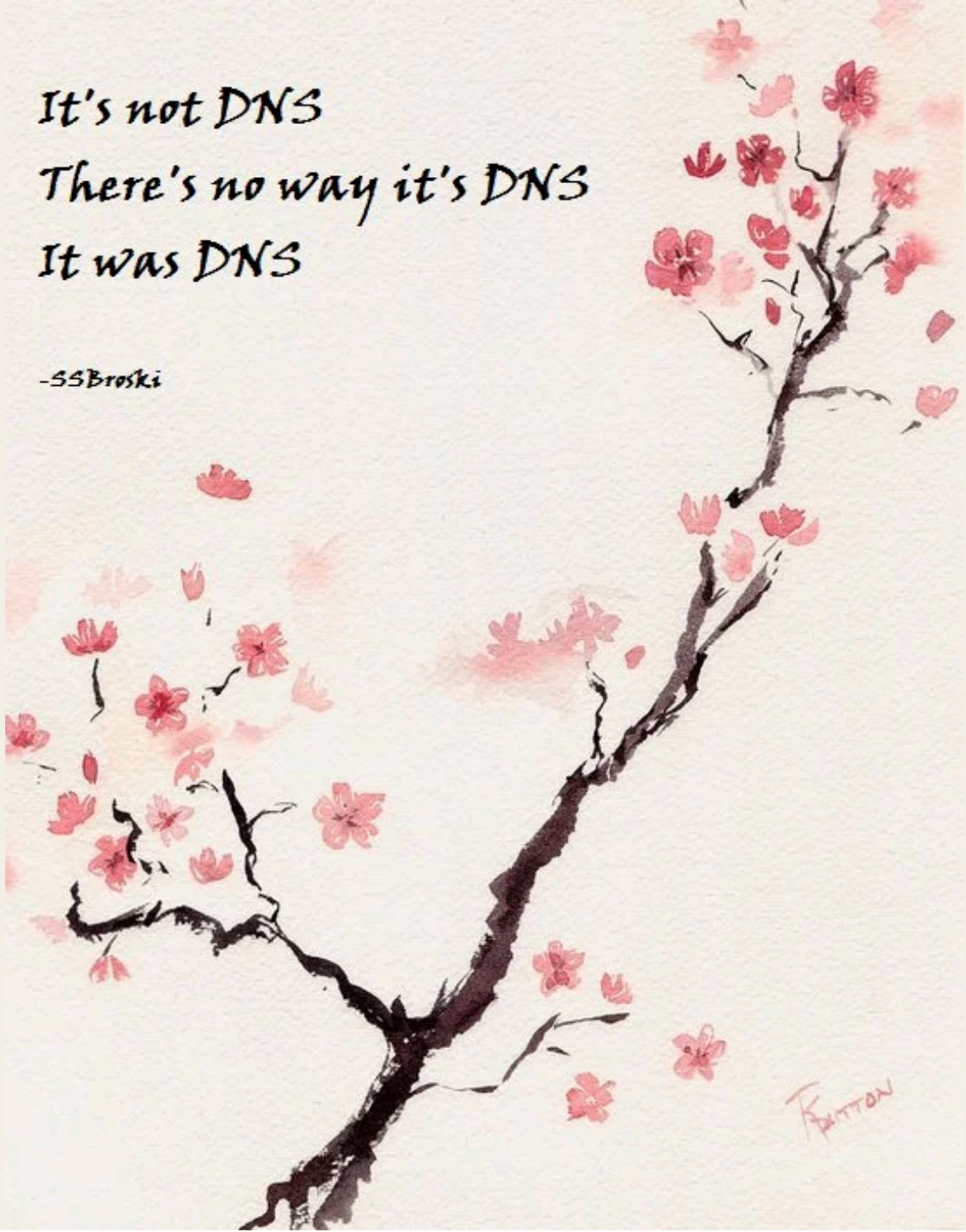


Agenda

1. **DNS Overview**
   What is the Domain Name System?

2. **DNS Security**
   How attackers abuse DNS, and how to defend it

3. **Wrap-up**
   What to remember



PART 1

DNS Overview

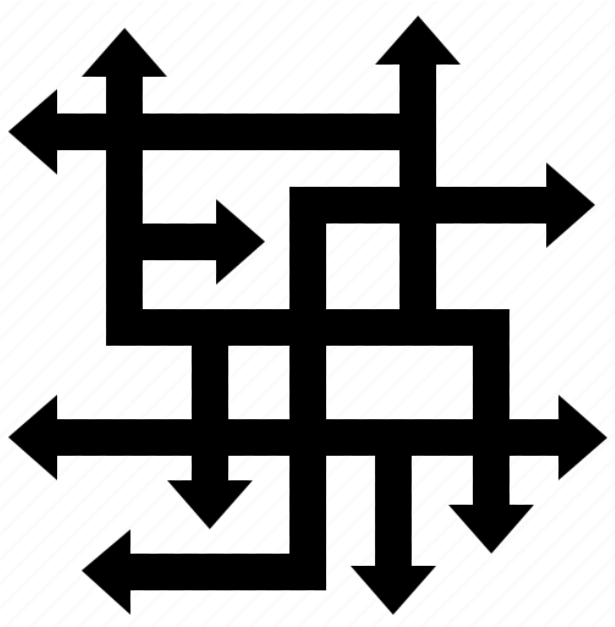
What is the Domain Name System?

DNS is hard ?

The only three hard problems in computer science are **cache invalidation** and **naming things**.

And **DNS is a caching system for names of things.**

(And off-by-one errors)

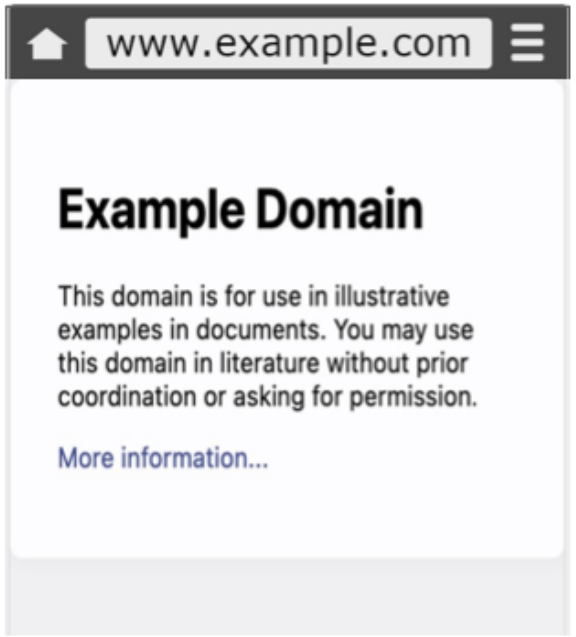


QUIZ

Before we start



What is happening behind the scenes ?



What do you know about the protocol allowing the name to be translated into technical information ?

Part 1: DNS Overview

1. How DNS is organized



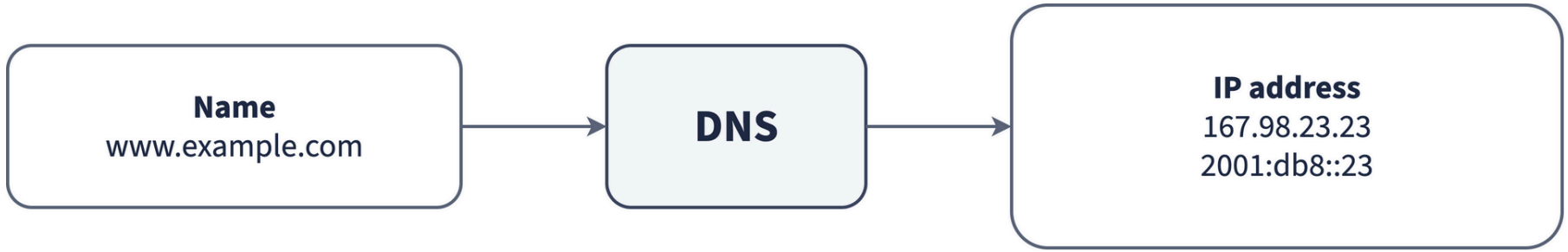
2. How a name is resolved

3. Advanced DNS concepts

What is DNS ?

DNS is a distributed, hierarchical system that translates names into the data needed to reach a service.

Website	Where is www.example.com? → its IP address
E-mail	Where do I deliver e-mail for example.com? → its mail servers
Address	Which name is behind this IP address? → a name



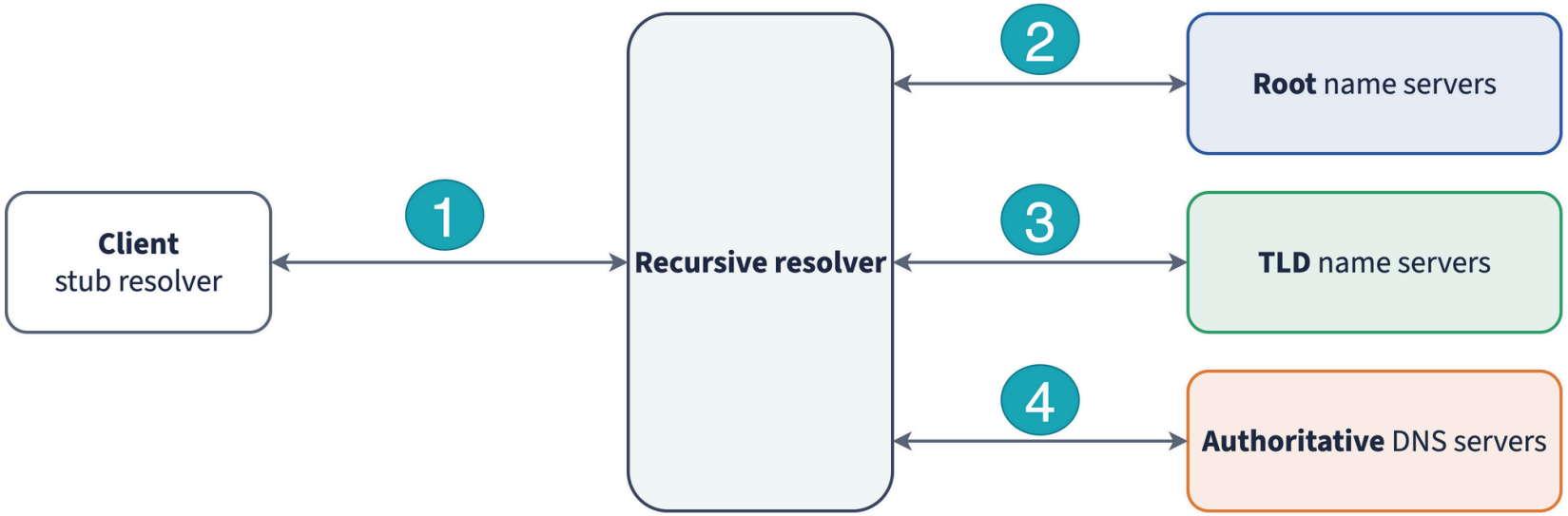
Critical infrastructure: when DNS fails, everything that uses names fails with it.

DNS Features

Globally Distributed	Multiple DNS servers managed by different operators
Loosely coherent	One global system, but caches can briefly keep old answers until their TTL expires
Scalable	The system can be scaled up and multiple servers can be added
Reliable	Failure tolerance due to hierarchical distributed architecture and local caches
Dynamic	Anyone can add domains and records without causing outage

DNS Protocol

- DNS is a client-server protocol
  - The **stub resolver** on your device asks for a **function** (IP, mail server …) of a **name**
  - The **recursive resolver**, a DNS server, finds the answer and responds with the record
- Requests and responses are normally sent via **UDP/53**
- Also uses **TCP/53** for large responses
  - Ex: copying all of a server's data to another server, answers too large for UDP (see EDNS later)



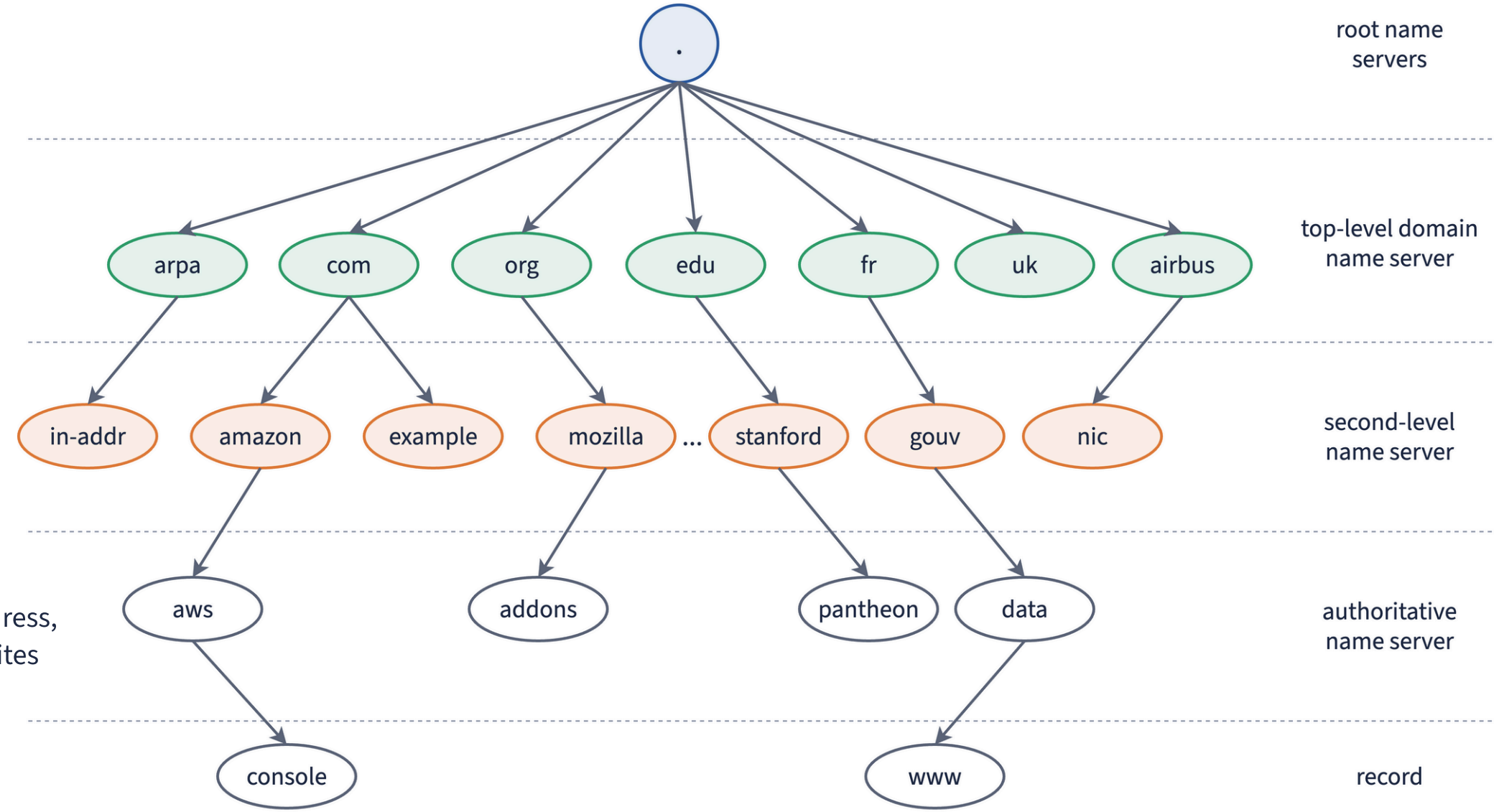
DNS Namespace Hierarchy

13 root server names, 2,048 instances (Nov 2026)

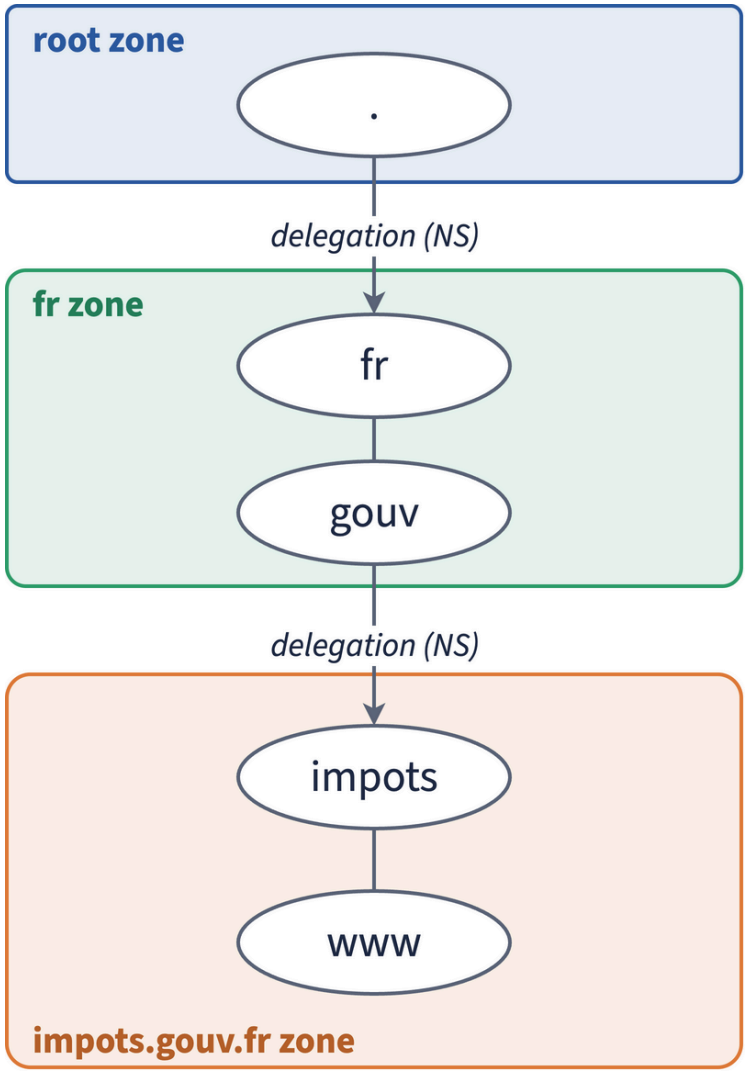
1436 TLDs (Nov 2026)

~400 million domains (Q2 2026)

Anycast: one IP address, served from many sites



Domain or Zone? Delegation



- A **zone** is the part of the tree managed as one unit. Its data is served by its **authoritative** name servers.
- A **delegation** is how a parent zone hands over a subtree, using NS records. This is the **referral** you will see in a resolution.
- A domain is not always a zone: gouv.fr has no NS records. It lives in the fr zone, which delegates impots.gouv.fr directly.
- **Glue:** dns1.impots.gouv.fr is inside the zone it serves. To break the loop, the parent also gives its address in the referral.

```
❯ dig NS gouv.fr +short
(no answer)
❯ dig NS impots.gouv.fr +short
dns1.impots.gouv.fr.
dns2.impots.gouv.fr.
```

Who is responsible for DNS ?

- **ICANN (Internet Corporation for Assigned Names and Numbers)**: Oversees the global DNS structure, including the policies governing domain name registration.
- **IANA (Internet Assigned Numbers Authority)**: Part of ICANN, IANA coordinates key technical aspects of DNS, including the root zone. It allocates IP address blocks and records top-level domains (TLDs). The root servers themselves are run by 12 independent operators.
- **Registry Operators**: Responsible for specific TLDs (like Verisign for .com), they maintain the records of domains under each TLD.
- **Registrars**: Accredited companies (e.g., GoDaddy) that allow individuals and organizations to register domain names.
- **ISPs and Network Admins**: Provide recursive resolvers, translating domain names to IP addresses for end-users.

Types of TLDs

COUNTRY CODE

ccTLD

`.fr` `.uk` `.ca` `.ae`
`.de` `.jp` `.io` `.tv`

Two letters, one per country or territory. Some are sold worldwide: .io, .tv.

IDN ccTLD

`.中国` `.pф` `.مصر`
`.한국` `.ελ`

Country codes in non-Latin scripts: Chinese, Cyrillic, Arabic, Korean, Greek.

GENERIC

Standard gTLD

`.com`
`.net`
`.org`
`.info`
`.biz`

Open to anyone, run under ICANN policy for the whole Internet.

New gTLD

`.airbus` `.google`
`.paris` `.bzh` `.app`

Brands, cities, regions and words since 2012. A 2026 round drew 1,615 applications.

Sponsored gTLD

`.aero` `.edu` `.gov`
`.coop` `.museum`

A sponsor decides who may register, for one community.

SPECIAL USE

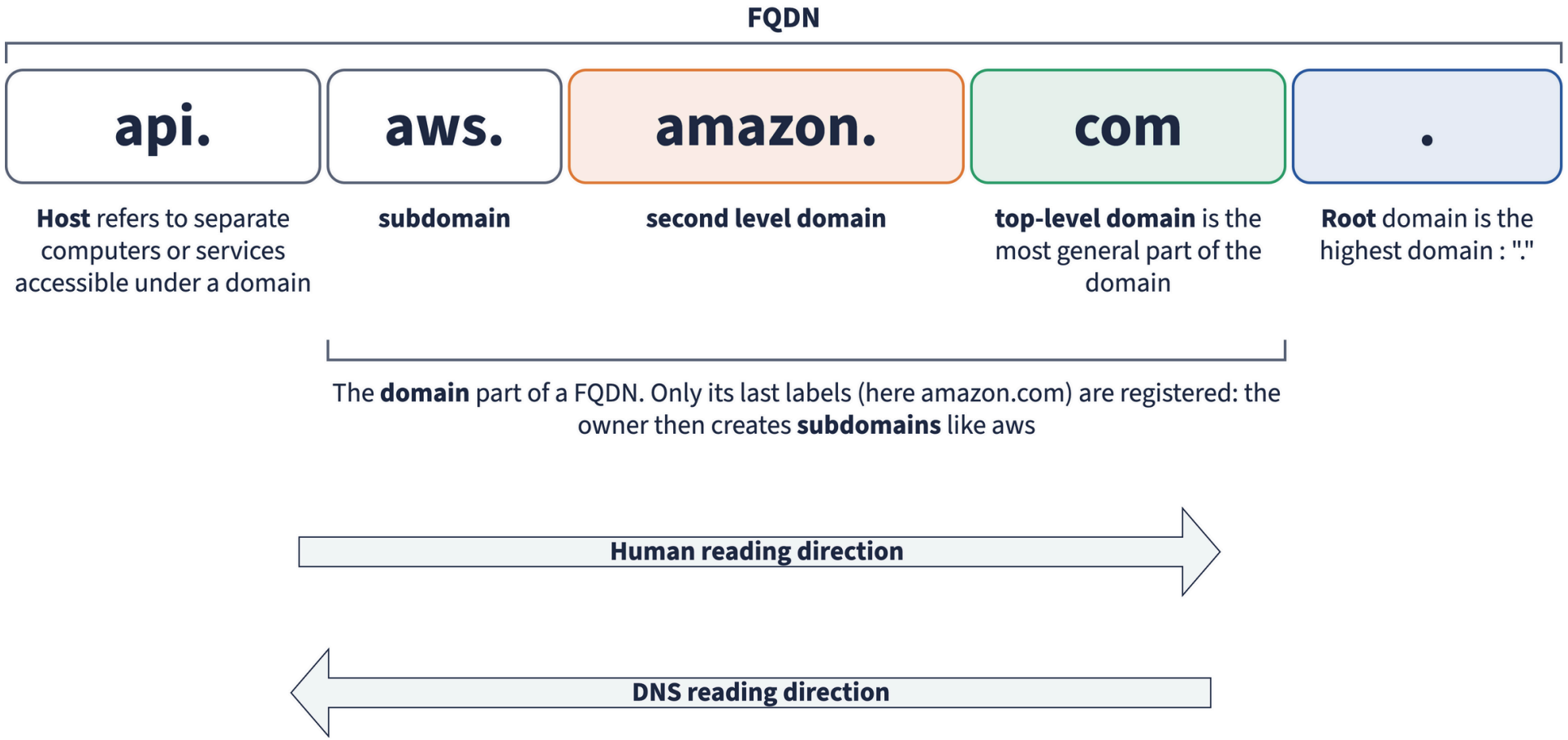
Special-use names

`.arpa`
`.local`
`.onion`
`.test`
`.example`
`.invalid`
`.localhost`

.arpa runs Internet infrastructure, such as reverse lookups. The others are reserved and never sold.

DNS Terminology: “name” decomposition

A **Fully Qualified Domain Name** is an absolute name containing the host and the domain. **A proper FQDN ends with a dot**, indicating the root of the DNS hierarchy.



Types of DNS Record

There are dozens of DNS record types (list here). The most common are:

A **AAAA**	Links the hostname to the IP address. (A: IPv4, AAAA: IPv6)
CNAME	Point a Domain Name to another Domain Name
TXT	Text records are used to store descriptive text (site verification, SPF/DMARC mail record, etc)
MX	Mail Exchanger record points to the email server responsible for the domain, with a priority.
SOA	identify the Start of Authority with essential parameters of the zone (primary server, contact info, TTL for negative answers)
NS	NameServer records identify the responsible DNS Server for answering this zone.
PTR	Pointer record links an IP address to a domain name (opposite to A)
SRV	SeRVice record points to a host and its corresponding port for a specific service with a priority. It is used for discovering a service.

CAA, LOC, HINFO, HTTPS/SVCB, RRSIG, NSEC, DNSKEY are also seen but are more “advanced ones”

QUIZ

Quiz time

Quiz round 1 · How DNS is organised



1. Open Dig Quest on your phone: **dns.jmanteau.fr**. Not in yet? Scan the code on the board, then pick a nickname.
2. Each question appears on your screen: answer before the timer ends.
3. Right and fast answers score points on the **Quiz** leaderboard.

Part 1: DNS Overview

1 How DNS is organized

2 **How a name is resolved**



3 Advanced DNS concepts

DNS Client: nslookup/dig

dig gives you the full detail, so it is the better tool for DNS investigation and testing.

nslookup was once marked as deprecated, but ISC later reversed that. It is almost universally available (e.g. built into Windows, where PowerShell also offers Resolve-DnsName).

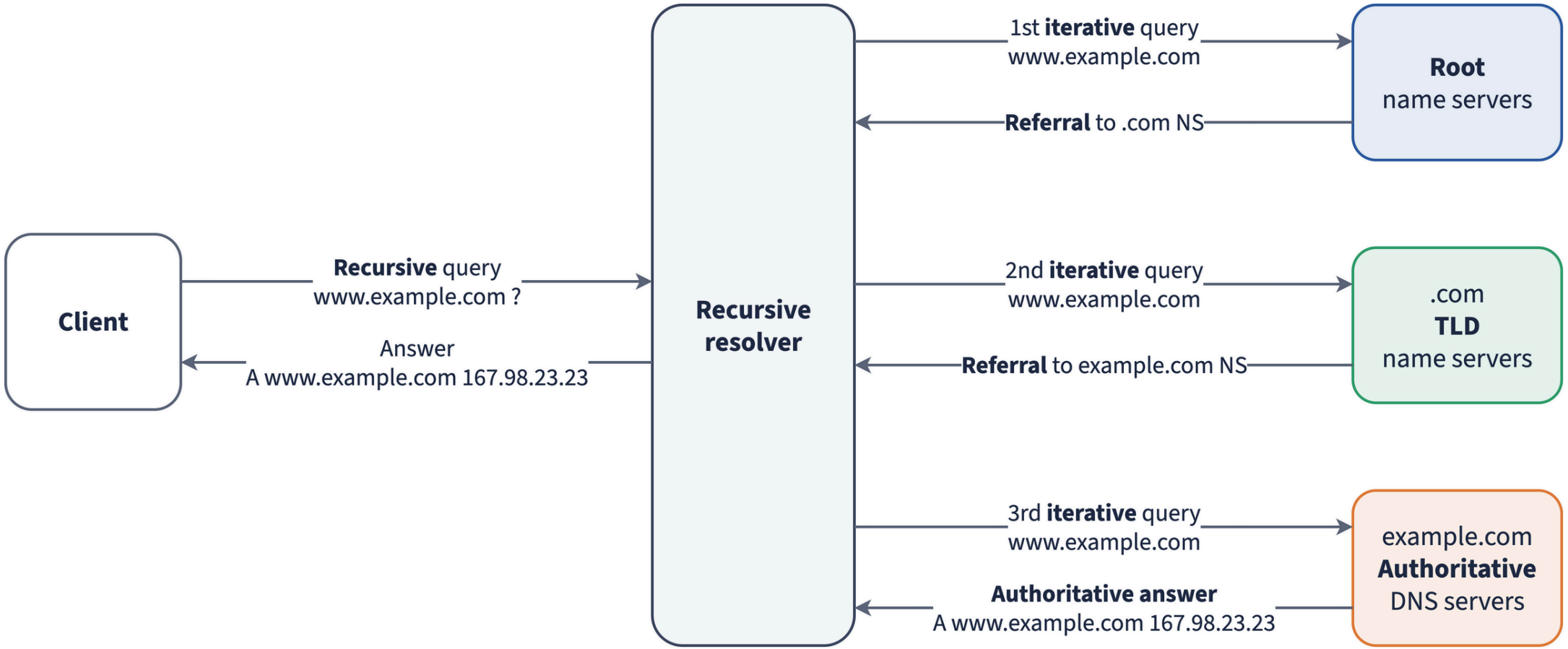
So my recommendation is:

- Master and have dig on your system for DNS investigation and testing.
- Know enough nslookup for basic troubleshooting on other devices

Live demo: reading a first dig answer

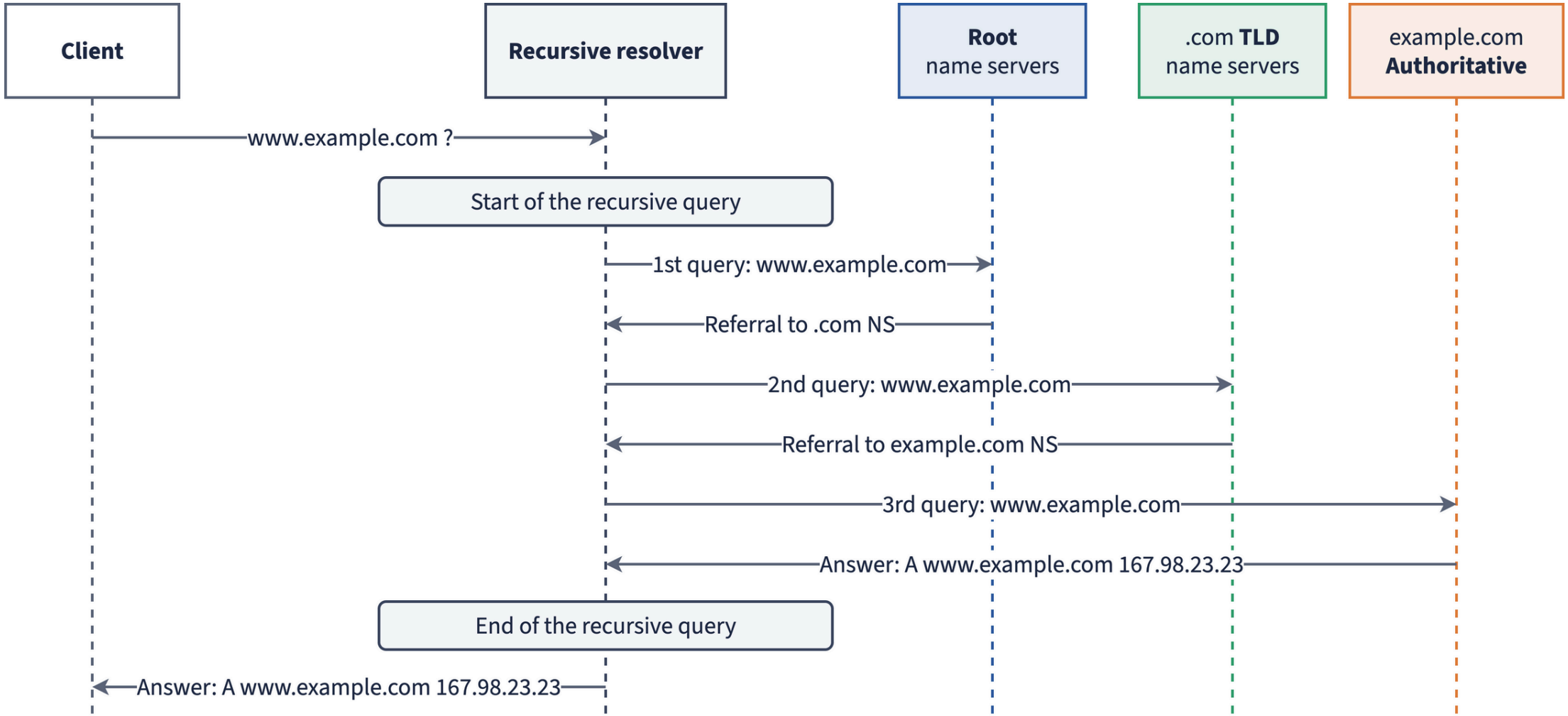
```
❯ dig www.example.com +noall +answer
name                 TTL   class  type   value
```

Full DNS Query and types of DNS Server



Think of a library: the recursive resolver is the librarian, the root an index pointing to a room, the TLD server a rack of books, and the authoritative server the dictionary holding the answer. A **forwarder** is a resolver that passes queries to another resolver instead of walking the tree itself.

Full DNSQuery



This is the classic view, with the full name sent at each step. With QNAME minimisation (RFC 9156), a resolver may send each server only the part it needs: better for privacy, same result.

Worked example: a CNAME restarts the resolution

```
❯ dig www.example.com +noall +answer
www.example.com.                    300    IN   CNAME   www.example.com-v4.edgesuite.net.
www.example.com-v4.edgesuite.net.   21600  IN   CNAME   a1422.dscr.akamai.net.
a1422.dscr.akamai.net.              20     IN   A       23.72.250.233
```

PREDICT **Starting from an empty cache: how many zones does the resolver have to visit?**

1. The recursive resolver reaches the example.com authoritative server: the answer is not an IP, but a **new name**.
2. New name, new zone: it **starts again** for www.example.com-v4.edgesuite.net (root, then .net, then edgesuite.net). Another CNAME.
3. It starts again for a1422.dscr.akamai.net (.net, then akamai.net). This time, an **A record**.
4. The client receives the whole chain in one answer. Each line keeps its own TTL.

Why so many steps? This site uses a **CDN** (Content Delivery Network): Akamai serves it from many locations and often protects it too. The CNAME hands the name over to the CDN's own zone.

LAB

Full DNS query: sequence diagram



Practical Exercise: Full DNS Query for www.impots.gouv.fr

Based on the sequence diagram (click on the link in its title), please recreate the full chain of the resolution of www.impots.gouv.fr

Tip: `dig +trace www.impots.gouv.fr` shows each step from the root.

```
❯ dig www.impots.gouv.fr +noall +answer
www.impots.gouv.fr. 129 IN  CNAME   www.impots.gouv.fr.edgesuite.net.
www.impots.gouv.fr.edgesuite.net. 21429 IN CNAME a1352.dscb.akamai.net.
a1352.dscb.akamai.net.  20  IN  A   95.100.133.81
a1352.dscb.akamai.net.  20  IN  A   95.100.133.74
```

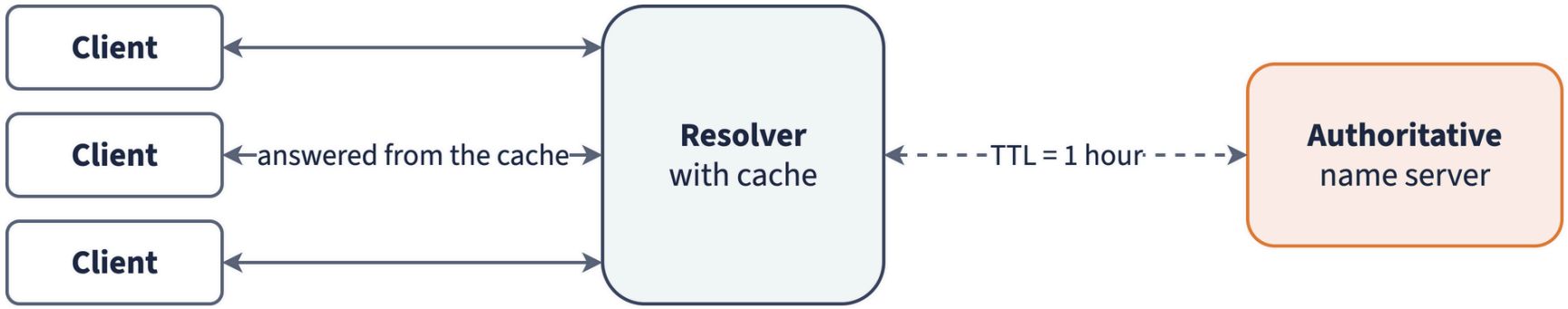
TTL: Definition

TTL (Time To Live) is a value configured on the DNS Server which informs the client / resolver **how long it should store the DNS record details before sending the query to get the updated information**. The DNS Server returns the TTL Value along the record.

Per the DNS RFC, DNS TTL values can be configured from 0 seconds to 24855 days (about 68 years).

However DNS client implementations can have different values.

This is why I personally advise TTL values between 1min and 1 day.



DNS Cache

DNS Cache (sometimes called a DNS resolver cache) is a temporary database, maintained by an OS, that contains records of all recent DNS lookups.

Before attempting to resolve a name, the OS will check if the entry is present in the cache. The entries are cleared according to their TTL (a resolver may still serve expired data if the authoritative servers are unreachable, RFC 8767). Before requesting, it will also check the “static cache”: the hosts file.

A browser will also have a DNS cache that it will use before using the OS cache (then the default recursive resolver if not in OS cache, OR its own configured resolver)

	Windows	Linux (if present)	macOS	Chrome	Firefox
Check Cache	ipconfig /displaydns	resolvectl show-cache (systemd 254+)	No simple listing command	chrome://net-internals/#dns (looks up one name)	about:networking#dns
Clear Cache	ipconfig /flushdns	resolvectl flush-caches	sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder	chrome://net-internals/#dns (Clear host cache)	about:networking#dns (Clear DNS Cache)

TTL: Lookup

```
❯ dig wikipedia.org +noall +answer
wikipedia.org.        403     IN     A     185.15.58.224
❯ dig ns wikipedia.org +noall +answer
wikipedia.org.        2907    IN     NS    ns0.wikimedia.org.
wikipedia.org.        2907    IN     NS    ns2.wikimedia.org.
wikipedia.org.        2907    IN     NS    ns1.wikimedia.org.
❯ dig wikipedia.org +norecurse @ns0.wikimedia.org +noall +answer
wikipedia.org.         600    IN     A     185.15.58.224
```

Resolver's remaining TTL

Locate and direct Query to authoritative NS

Authoritative TTL answer

PREDICT **Same record, asked directly to the authoritative server: will the TTL be 403, lower, or higher?**

If your default DNS server is not the authoritative server for the zone you are digging (so 99,99% of the time) dig will show the time remaining (until the next refresh) instead of the raw TTL value in this position.

Reading a dig answer: the sections

```
❯ dig www.example.com
; <<>> DiG 9.20.10 <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ⟶»HEADER«⟵ opcode: QUERY, status: NOERROR, id: 17148
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: e2abe7f2209ce7a5… (good)
;; QUESTION SECTION:
;www.example.com.                IN   A
;; ANSWER SECTION:
www.example.com.                300    IN   CNAME   www.example.com-v4.edgesuite.net.
www.example.com-v4.edgesuite.net. 21600  IN   CNAME   a1422.dscr.akamai.net.
a1422.dscr.akamai.net.          20     IN   A       23.72.250.233
a1422.dscr.akamai.net.          20     IN   A       23.72.250.203
;; Query time: 185 msec
;; SERVER: 192.168.227.200#53(192.168.227.200) (UDP)
;; WHEN: Tue Nov 25 09:56:07 CET 2025
;; MSG SIZE  rcvd: 185
```

- **Header: status, flags, and how many records in each section**
- **EDNS: room for answers over 512 bytes**
- **Question: the name, class and type you asked**
- **Answer: the records, each line with its own TTL**
- **Footer: which server answered, how fast, over UDP or TCP**

Reading a dig answer: status and flags

Status	Meaning
NOERROR	The query worked
NXDOMAIN	This name does not exist
NODATA	The name exists, but has no record of this type. Not a real status: NOERROR with 0 answers
SERVFAIL	The server could not give a valid answer (e.g. DNSSEC failure)
REFUSED	The server declines to answer (policy)

Flags

- `aa` authoritative answer
- `rd` recursion desired
- `ra` recursion available
- `tc` truncated
- `ad` validated by DNSSEC

PREDICT **impots.gouv.fr has no IPv6 address. Asking for AAAA gives NOERROR, NXDOMAIN or SERVFAIL?**

```
❯ dig @1.1.1.1 AAAA impots.gouv.fr
;; status: NOERROR      ;; flags: qr rd ra      ANSWER: 0     (this is NODATA)
impots.gouv.fr.   2908   IN   SOA   dns0.dgfip.finances.gouv.fr. ... 3600
```

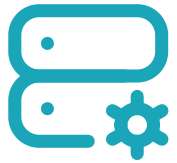
Negative caching: “this does not exist” is cached too, so the same lookup is not repeated. The last SOA field sets the limit (here 3600 s), and 2908 s remain for this resolver. So a record you just created can stay invisible for a while.

Part 1: DNS Overview

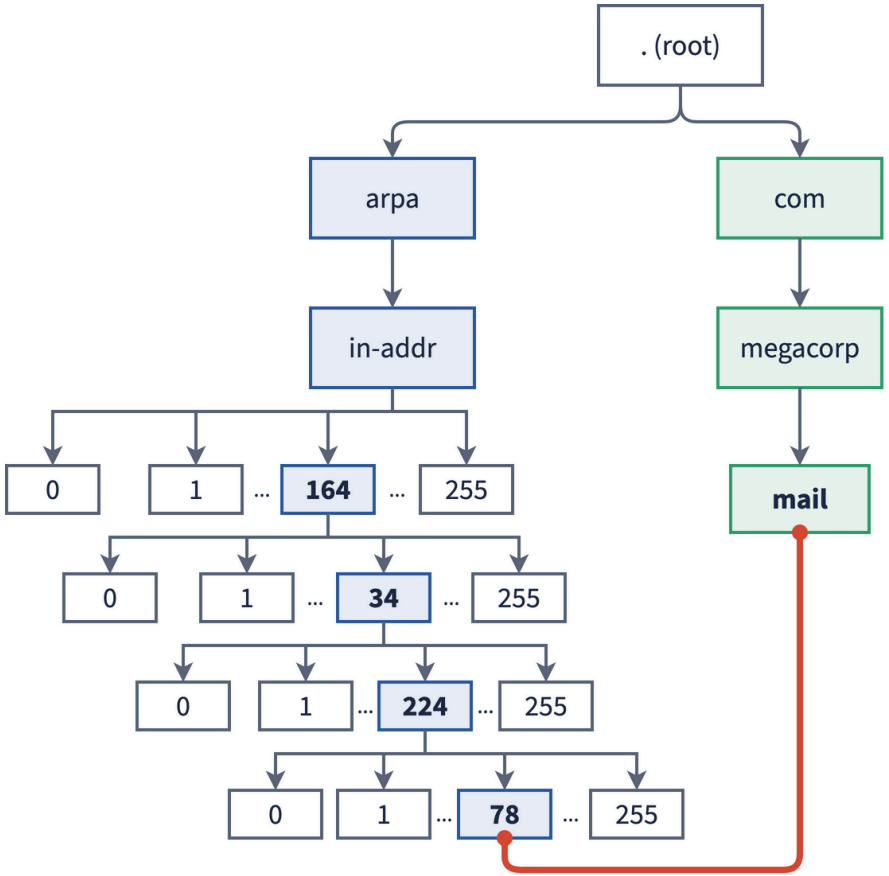
1 How DNS is organized

2 How a name is resolved

3 Advanced DNS concepts



Reverse lookup and PTR records



Goal: find the name behind an IP address. Logs, mail servers and traceroute use it to show names instead of numbers.

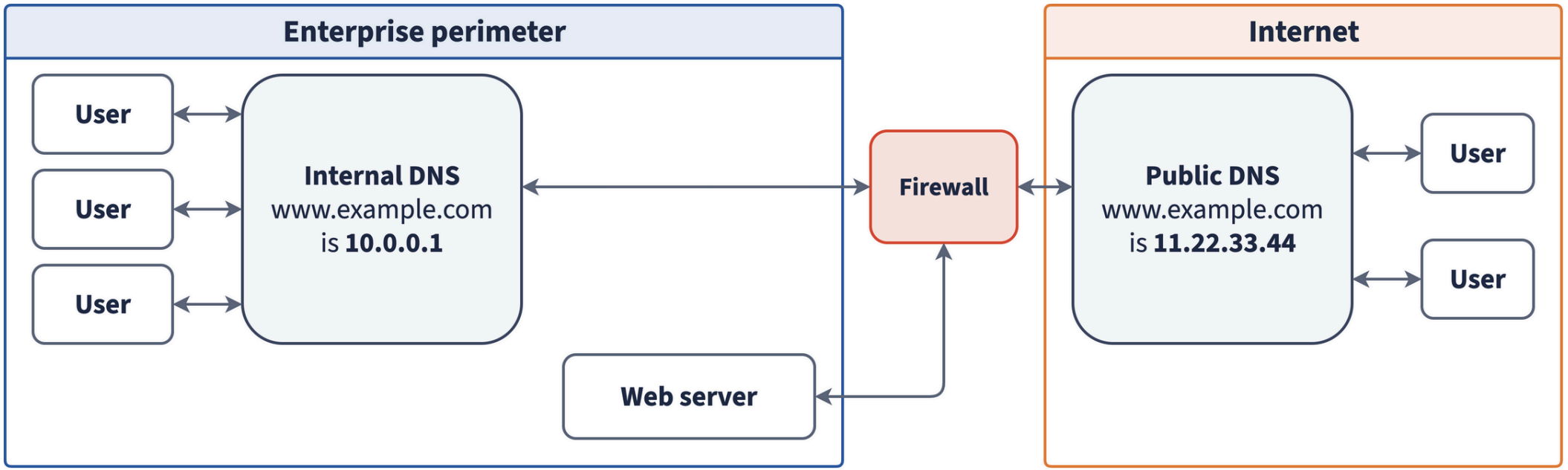
How: DNS can only look up names, so the address becomes one. Its numbers are written backwards under `in-addr.arpa`, and a **PTR** record there holds the answer.

```
❯ dig -x 164.34.224.78 +short
mail.megacorp.com.
stored as: 78.224.34.164.in-addr.arpa.
           PTR mail.megacorp.com.
```

Why backwards? A name is read from right to left: `com`, then `megacorp`, then `mail`. An IP address is read the other way: `164` (the network) first, `78` (the machine) last. Flipped, it reads like a name, and the tree starts with `164`.

The network then sits at the **end** of the name. DNS delegates by name ending, so the owner of `164.34.x.x` gets its own reverse zone, `34.164.in-addr.arpa`, just like a domain.

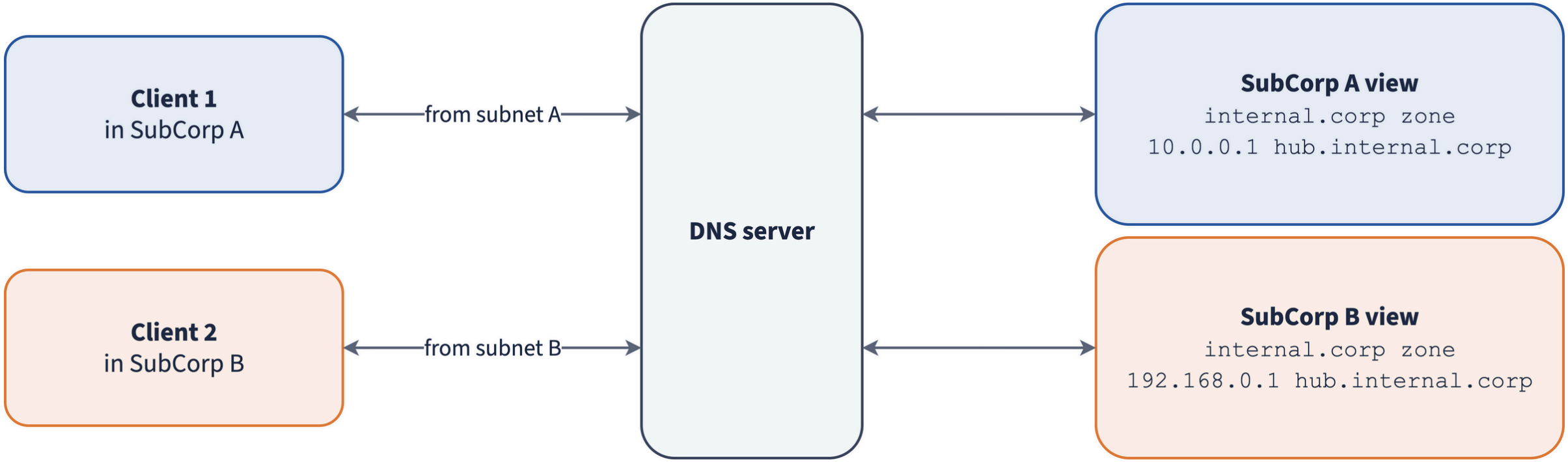
DNS Concepts: Split Horizon



Split Horizon is the DNS concept that the same name gets a different answer depending on where the resolution is done. It can be implemented in different ways:

- DNS Views
- Different DNS servers (hence different zones)
- DNS Global Load Balancing
- RPZ (seen in the next part)

DNS Concepts: Views



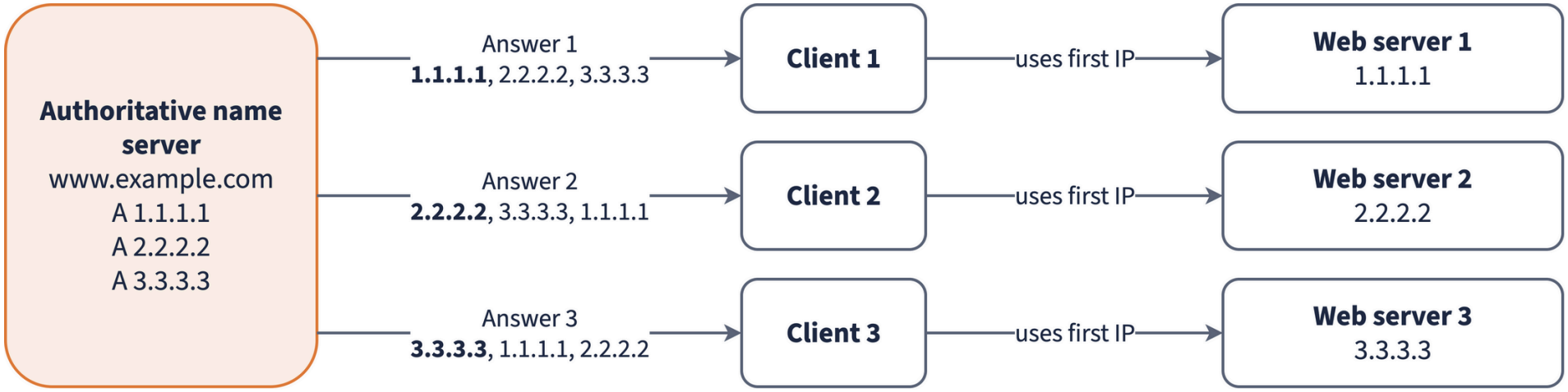
The DNS server matches the client subnet and chooses the right view to answer from. This is simple here because the server holds both versions of the zone itself.

EDNS: bigger answers, and the client's location

- **EDNS(0)** extends the DNS message. A client can announce that it accepts more than the original 512 bytes over UDP. DNS Flag Day 2020 recommended 1232 bytes, to avoid IP fragmentation.
- An answer that does not fit comes back truncated (`tc` flag), and the client retries over TCP.
- **Client Subnet (ECS)** is an EDNS option: the resolver tells the authoritative server which network the client is on, so it can answer with a nearby server. It also shares part of the client's address with that server.

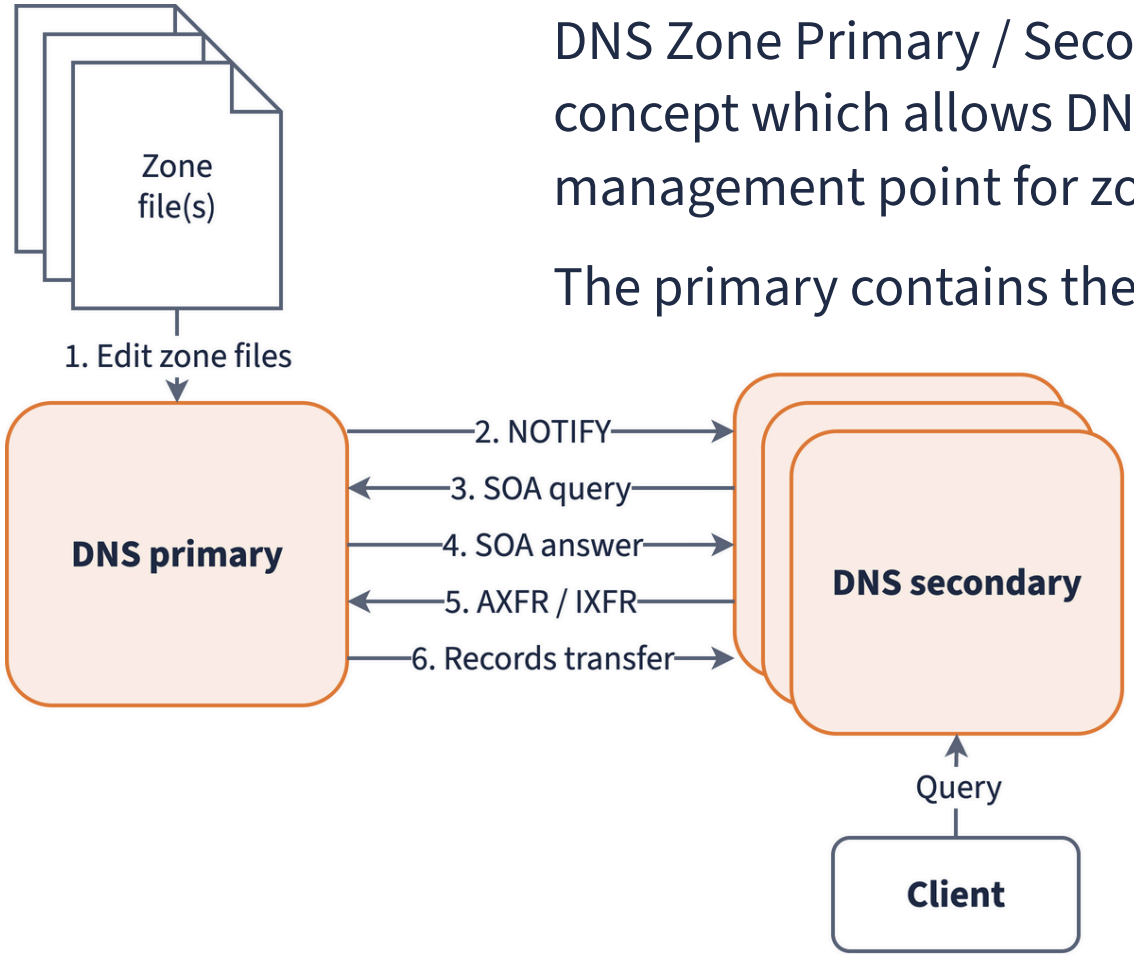
```
❯ dig @8.8.8.8 www.wikipedia.org +subnet=129.20.0.0/24 +short     # a network in France
dyna.wikimedia.org.
185.15.58.224
❯ dig @8.8.8.8 www.wikipedia.org +subnet=138.186.8.0/24 +short    # a network in Chile
dyna.wikimedia.org.
195.200.68.224
```

DNS Concepts: Round Robin



- **What it is**: A DNS load balancing method managed by the authoritative nameserver, without needing dedicated load-balancing hardware.
- **Use Cases**: Ideal for websites or services hosted on multiple redundant servers or when a single DNS name directs to multiple proxy servers.
- **How it works**: The nameserver returns all the server IPs and rotates their order in each response. Clients mostly use the first one (shown in the diagram), which spreads the traffic.
- **Limits**: Caching makes the balancing rough, and a server that is down keeps being handed out.

DNS Concepts: Primary, Secondary & Zone Update



DNS Zone Primary / Secondary (used to be called Master / Slave) is a concept which allows DNS servers to scale while keeping one unique management point for zone edition.

The primary contains the DNS data source of truth.

The secondary reads the primary's SOA record, checks the serial number and requests a zone transfer if it is higher

It is impossible to determine from a query result that it came from a zone primary or secondary.

LAB

Request DNS with dig



Practical Exercise: Request DNS with Dig

You now have what the questions need: records, resolution, TTL, reverse DNS and split horizon.

The lab is in `dns-dig-lab`, with a one-page **dig cheat sheet**. Then check your answers and compete on **Dig Quest** at **dns.jmanteau.fr**: scan the code on the leaderboard.

Course site

Cheat sheet

QUIZ

Quiz time

Quiz round 2 · Resolution, caching and advanced concepts



1. Open Dig Quest on your phone: **dns.jmanteau.fr**, same session as the dig lab. Not in yet? Scan the code on the board.
2. Each question appears on your screen: answer before the timer ends.
3. Right and fast answers score points on the **Quiz** leaderboard.

Agenda

1 **DNS Overview**
What is the Domain Name System?

2 **DNS Security**
How attackers abuse DNS, and how to defend it

3 **Wrap-up**
What to remember



PART 2

DNS Security

How attackers abuse DNS, and how to defend it

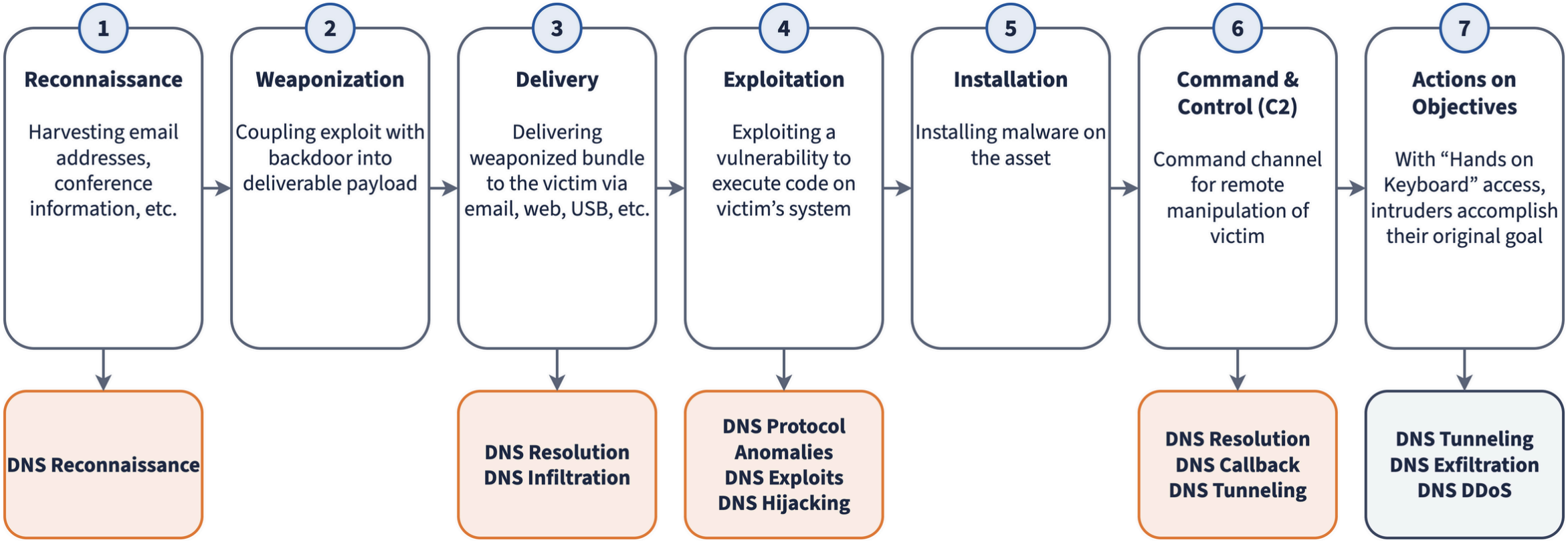
From Part 1 to Part 2: why attackers like DNS

- **DNS is allowed out** of almost every network, because everything depends on it.
- **The recursive resolver delivers any query** to the authoritative server of the domain, even if an attacker runs that server.
- **A name can carry data**: `<data>.s.attacker.com` reaches the attacker, and the answer can carry data back.

```
Infected client → Enterprise resolver → Root, TLD → Attacker's server

The resolution you just traced, with a different destination.
```

DNS in the kill chain



The kill chain (Lockheed Martin) lists the stages of a cyberattack. DNS plays a role in most of them.

Part 2: DNS Security

1 **How attackers abuse DNS**

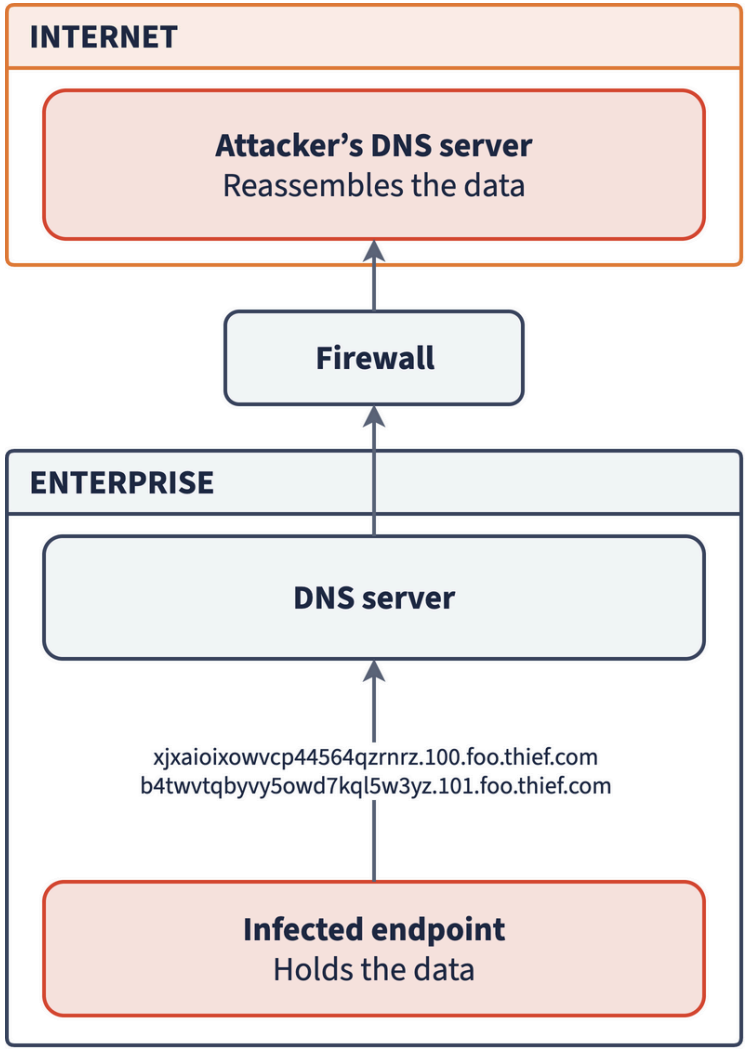


2 Detecting and blocking abuse

3 Protecting DNS itself

DNS as a transport: exfiltration

- Attacker registers a domain & sets up an authoritative DNS server on the Internet to act as the tunnel endpoint
- Data to be sent from inside network is:
  - Encrypted using public key
  - Encoded into a-z, 0-9 and – using algorithm such as Base32, which allows up to 110 bytes to be encoded into an FQDN
  - Divided into chunks of up to 63 characters (label limit)
  - Sent as individual queries in format of <chunk>.domain
- Queries can be sent direct (if allowed by firewall), via a network's existing DNS servers or via proxies (e.g. request http://<chunk>.domain)
- If sent directly, the source address can be spoofed to make tracing harder
- Attacker's authoritative server receives encoded chunks, reassembles data, decodes & decrypts using private key



DNS as a transport: infiltration

- Can be used to download a payload in a covert way for example.
- TCP features can be imitated by encoding the chunks with additional data, such as Checksum & Packet Number
- Data can be sent back in a variety of records, e.g.
  - A - allowing 4 bytes (enough for codes, e.g. 1.1.1.200 = resend packet 200)
  - AAAA - allowing 16 bytes
  - MX record : 2 bytes + domain name (255 bytes)
  - CNAME - allowing up to 110 bytes in Base32
  - TXT - allowing N x 220 bytes in Base64 (up to RDATA limit of 64kB)
  - NULL - allowing up to 65,535 bytes in the record, in practice limited by the DNS message size
- Using TXT and NULL make transmission faster, at expense of easier detection

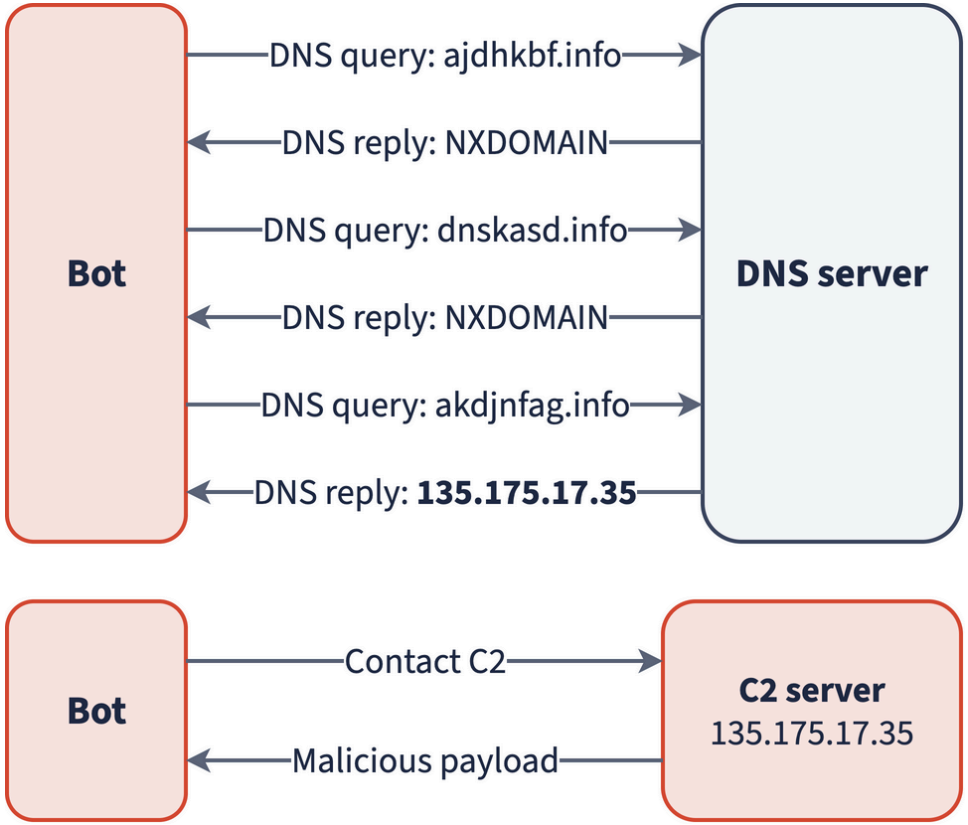
DGA – Domain Generation Algorithm

An algorithm producing Command & Control (C2) rendezvous points dynamically

For example: every day malware connects to time-based server FQDN: *<month>-<day>-<year>.com* i.e. on December 24, 2017 malware connects to 2017-12-24.com

Why? The attacker registers just one of today's domains, while defenders would have to block them all.

Family	Example domain
DirCrypt	vlbqryjd.com
Bamital	b83ed4877eec1997fcc39b7ae590007a.info
CCleaner	ab6d54340c1a.com



C2 vocabulary

C2 (Command and Control) is how an attacker remotely drives the malware on a victim's machine.

- **Implant**: the software placed on the target to keep access and take orders
- **Beacon**: the implant regularly checks in with the C2 server for instructions
- **Jitter**: a random delay added between check-ins, to make the pattern less predictable
- **Fast flux**: hiding C2 servers behind rapidly changing addresses, using the short TTLs and round robin you already know

The next lab uses **Sliver**, an open-source C2 framework. Its implant talks to its server only through DNS queries.

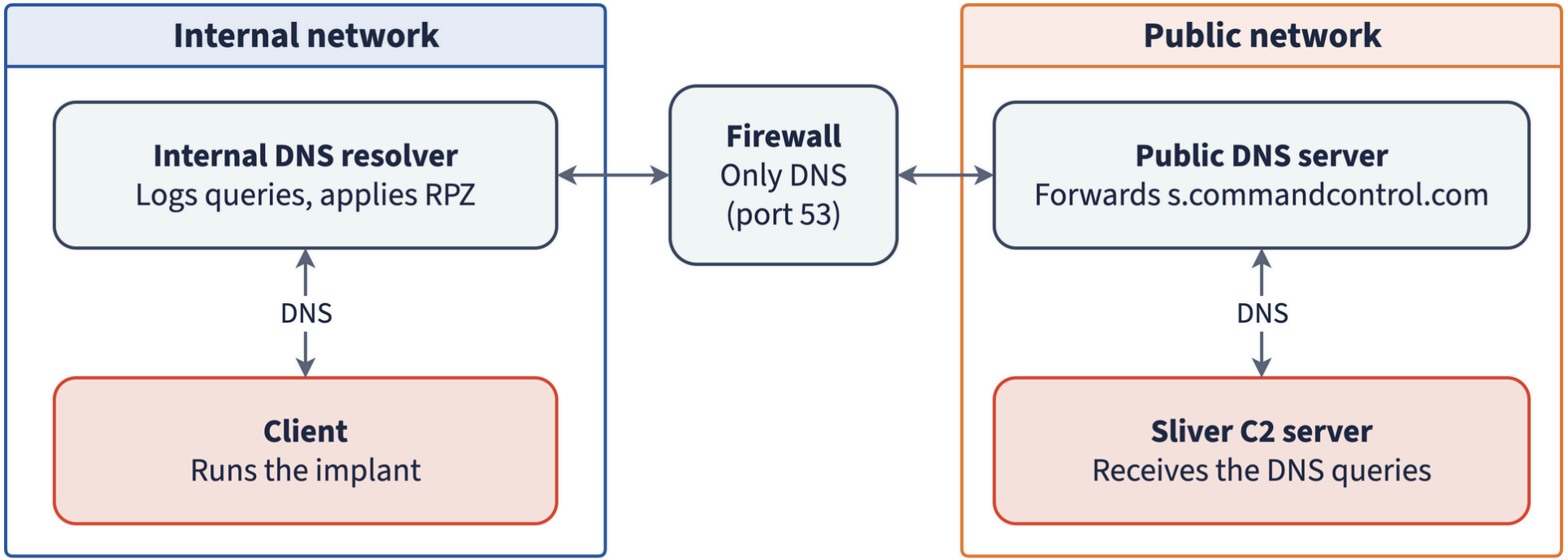
LAB

Command and control over DNS



Course site

Practical Exercise: DNS C2 with Sliver (Red Team)



Red team: client and C2 server **Blue team:** internal DNS resolver and firewall

Part 2: DNS Security

1 How attackers abuse DNS

2 **Detecting and blocking abuse**



3 Protecting DNS itself

Lookalike Domain Detection

- Detects from Newly Observed Domains (NOD), domain names that try to mimic popular domains (top-100 lists) or strategic domains for phishing (e.g. banks) and spamming.
- Detects domains that overlaps and/or have homograph n-grams, e.g. letter ‘O’ and number ‘0’ are considered homographs. Use distance analysis to detect the likelihood of lookalike attacks
- Detects
  - Letter replacement, example g00gle.com, gooogle.com, bankofthevvest.com, rn1cr0soft.com

    apple.com >>> appIe.com (Arial) apple.com >>> appIe.com (Cutive)
  - TLD change, example, walmart.cc
  - other generic highly resembled domains, example: bankofAmericas.com
- SOA record is analyzed and used for catching false positives.

EXERCISE

Which of these are malicious?

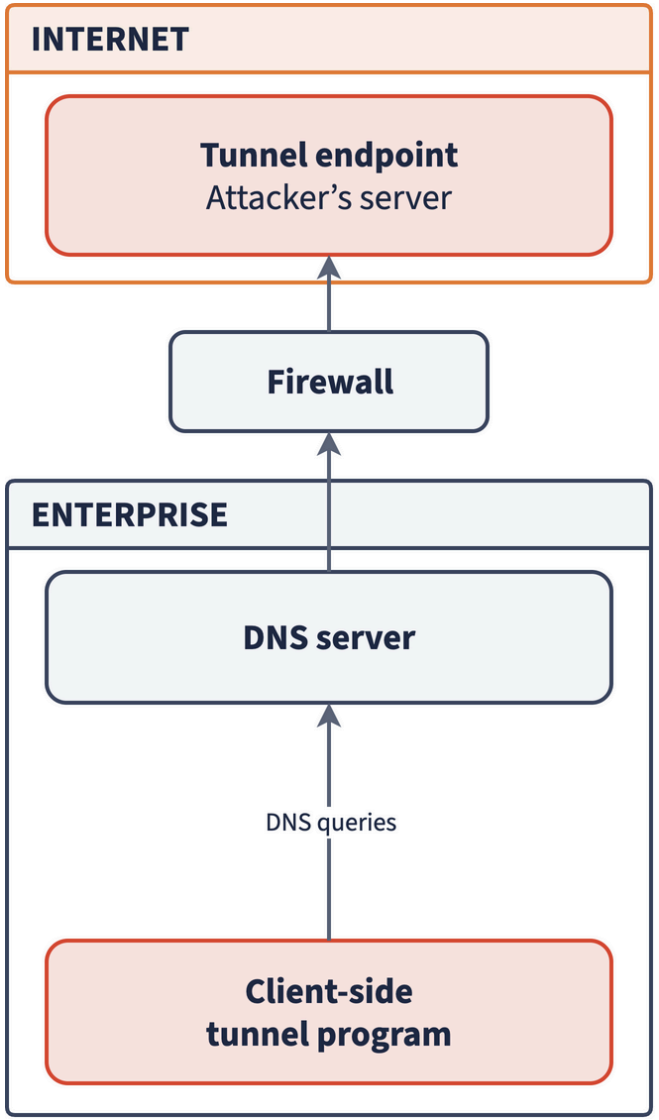
- g63uar2ejiq5tlrkg3zezf2fkemc6pi88tz.er.spotify.com
- a-0.19-b3000081.a010083.15e0.1d99.36d4.210.0.ic7arfsqqzf694fs8zf8nz2t9b.avts.mcafee.com
- 4rzjp8zy7i7vawluximoxrko1p2tn58gj0fjjj2g.p.03.s.sophosxl.net
- p2.a22a43lt5rwfg.ihg5ki5i6q3cfn3n.191742.i1.ds.ipv6-exp.l.google.com
- a1294.w20.akamai.net
- dnn506yrbagrg.cloudfront.net
- A98dc034c7781a941ebabac02262202668bbe918ea9fb5289cd2.r58.cf2.rackcdn.com

None of them. CDNs, anti-virus and other services legitimately encode data in DNS names, so detection has to cope with these false positives.

Detecting tunnels: the traditional approach

- Use a firewall to **rate limit UDP**, slowing the speed at which data can be transferred but not preventing it from happening
- **Analyze logs by hand** / using a visualization tool to find evidence of tunneling / data exfiltration after it's happened. Typically achieved by looking for abnormal query types (such as NULL), particular REGEXs and long queries all of the same length.
- **Tunnel endpoints can then be blocked** at the firewall level. Provides protection only if the attacker continues to use the same domain/server in the future.
- **Manually configure security appliances** (such as DDoS mitigation platforms) to block specific REGEXs, query types, long queries etc. Laborious process that risks blocking legitimate software such as anti-virus, that use DNS to check for new signatures, file hashes etc

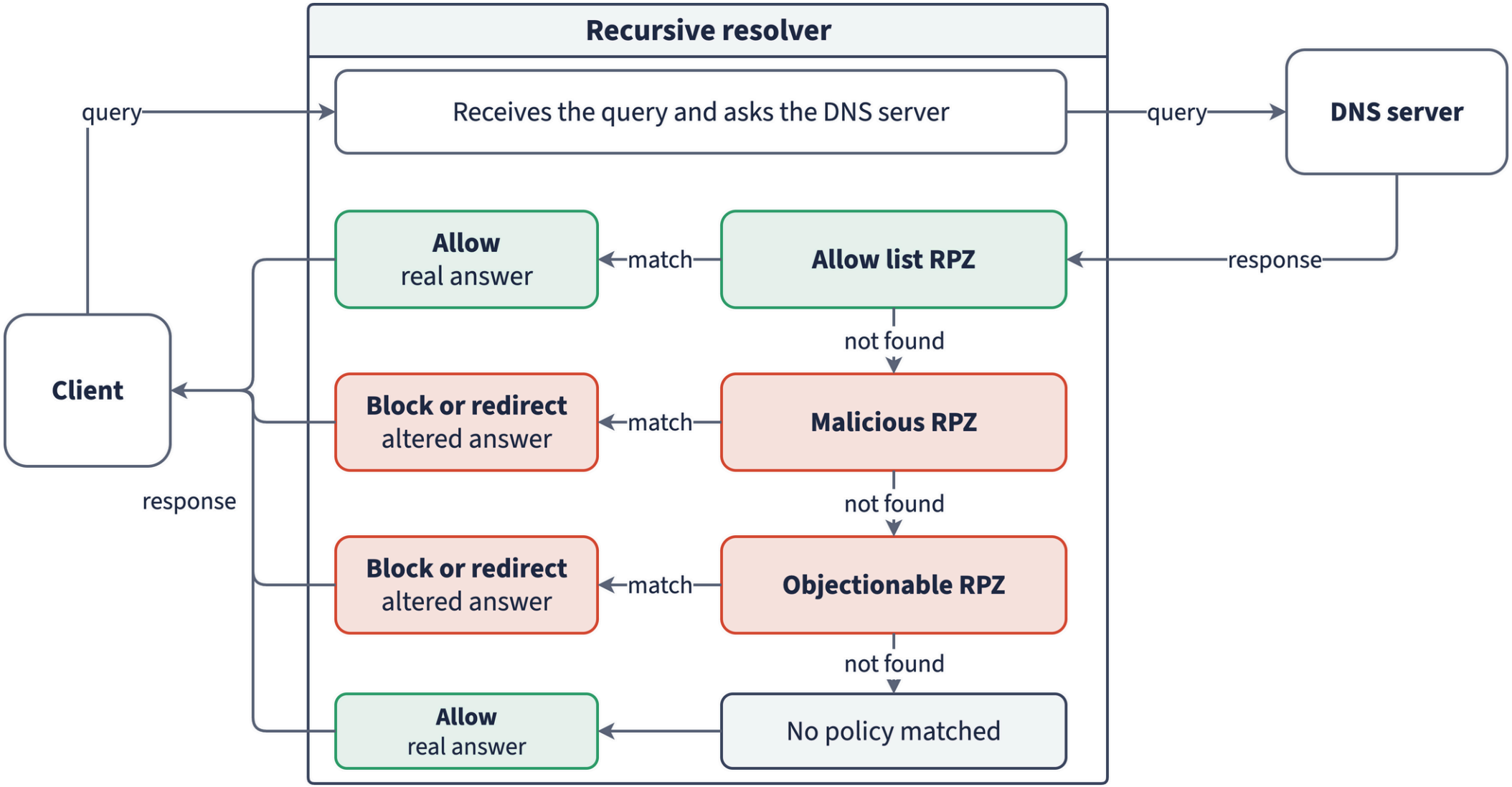
Ineffective against APTs & software that can be configured to vary query length, query type etc. to avoid detection



DNS Concepts: RPZ definition

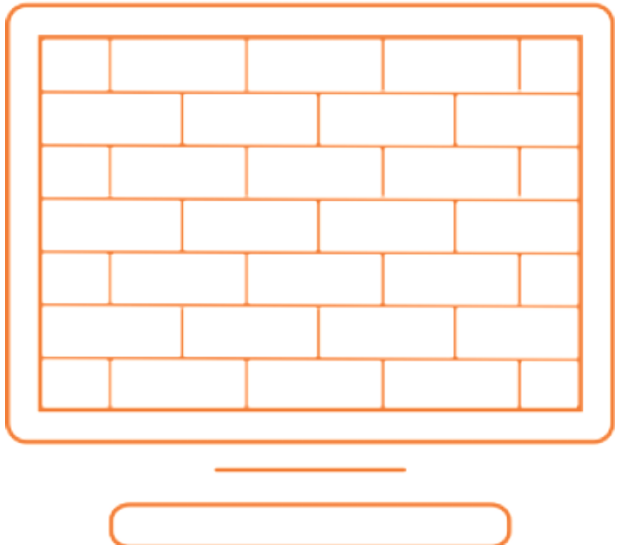
- DNS Response Policy Zone (RPZ) is a DNS zone which allow to customize policy in DNS servers, so that the server returns modified answers to clients' queries.
- **RPZ** provides a way to **alter a DNS response** in real time. RPZ is a "lying" mechanism.
- It can be used to modify potentially unsafe DNS Data to block communication or provide local data to redirect to a "walled garden".
- RPZ is the mechanism. A **DNS firewall** (also DNS sinkhole, or **Protective DNS**) is the usual way to use it: altering or blocking answers for chosen domains.

DNS Concepts: RPZ schema

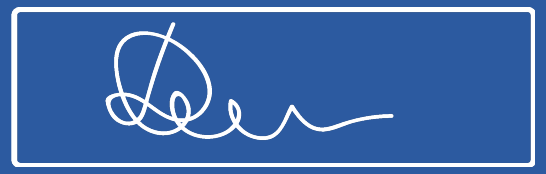


DNS Firewall

- A DNS firewall is RPZ in use: like a firewall, it implements a security policy
- Has triggers:
  - Queried domain name
  - IP address in response
  - IP address of querying client
- And actions:
  - Reject (NXdomain)
  - Block (no data)
  - Redirect
  - Passthrough and log



Detecting tunnels: DNS firewall products



Signature

Use **algorithms** to detect and block known DNS tunneling tools, at full network speed



Reputation

Use **DNS Firewall** to automatically block known tunnel, C2 and data exfiltration endpoints and newly registered domains using a threat intelligence database



Behavior

Use **DNS heuristics** to detect and block zero day data exfiltration, using real-time streaming analytics to mitigate against previously unseen code

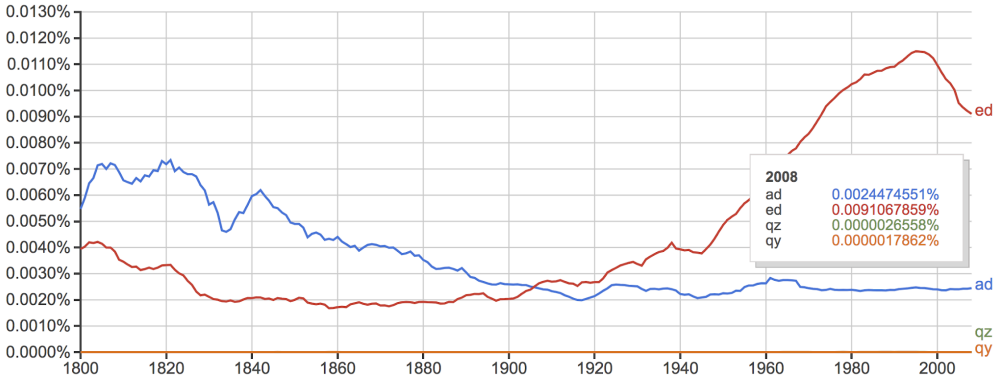
DNS heuristics

- **Entropy**
  - Higher Entropy => more information transferred
  - Legitimate DNS names often have dictionary words or something that looks meaningful. Encoded names have a higher entropy. DNS names that have high entropy can be an indicator of tunneling.
- **Lexical**
  - Analysis of individual characters in domain names
  - non-letters (numbers or allowed special characters) character ratio
  - hex /A-F/ character ratio
  - vowel character ratio

DNS heuristics

- **N-Gram**
  - Detects non human like domain names based on character distribution. Focus is on 2- and 3-gram (i.e. sequences of 2 or 3 characters, or bigram and trigram analysis).





Source: https://books.google.com/ngrams

DNS heuristics

- **Gini index** and **classification error** – two other measures of how evenly the characters are spread
- **Number of Labels** – number of domain labels in an FQDN payload
- **Frequency**: a resolver asking the same server again and again is normal. The signals are many unique subdomains under one domain, a high NXDOMAIN rate, and queries repeated at precise intervals
- **Size** Higher payload size => more information transferred

DNS heuristics: real numbers

Name	What it is	Length	Entropy	Gini	Class. error
`wikipedia`	Chosen by a human	9	2.64	0.81	0.67
`facebook`	Chosen by a human	8	2.75	0.84	0.75
`vlbqryjd`	DGA (DirCrypt)	8	3.00	0.88	0.88
`ab6d54340c1a`	DGA (CCleaner)	12	3.25	0.89	0.83
`b83ed4877eec1997fcc39b7ae590007a`	DGA (Bamital)	32	3.59	0.91	0.84
`xjxaioixowvcp44564qzrnrz`	Tunnel chunk	24	3.86	0.92	0.88
`g63uar2ejiq5tlrkg3zezf2fkemc6pi88tz`	**Legitimate (Spotify)**	**35**	**4.29**	**0.95**	**0.91**

PREDICT **Which name has the highest entropy, the most “random-looking” letters?**

Worked example, facebook: 6 letters appear once and “o” twice, so entropy = 6 × (1/8 × 3) + 1/4 × 2 = **2.75** bits per character.

The legitimate name scores highest. No single indicator is enough: this is why the lab’s script combines many into one score.

LAB

Detect and block DNS C2



Practical Exercise: DNS C2 with Sliver (Blue Team)

Back to the lab you left running. Now you are the defender:

- Find the C2 traffic in the DNS logs, using the indicators you just saw
- Tune the detection, to remove false positives
- Block the C2 domain with RPZ, and check that the implant loses contact

Course site

QUIZ

Quiz time

Quiz round 3 · DNS security



1. Open Dig Quest on your phone: **dns.jmanteau.fr**, same session as the dig lab. Not in yet? Scan the code on the board.
2. Each question appears on your screen: answer before the timer ends.
3. Right and fast answers score points on the **Quiz** leaderboard.

Part 2: DNS Security

1 How attackers abuse DNS

2 Detecting and blocking abuse

3 Protecting DNS itself



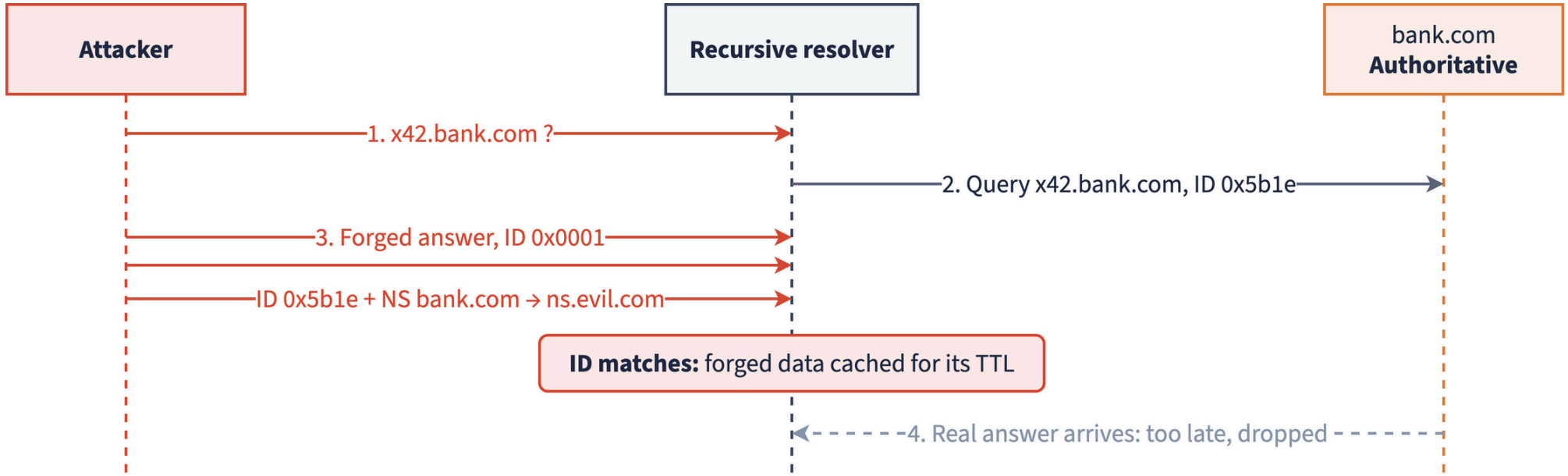
Types of DNS attacks: volumetric/DDoS

Attack	What it does	Main defense
DNS reflection/DrDoS attacks	Using third-party DNS servers (mostly open resolvers) to propagate a DoS or DDoS attack	Close open resolvers, block spoofed sources
DNS amplification	Using a specially crafted query to create an amplified response to flood the victim with traffic	Response rate limiting
TCP/UDP/ICMP floods	Denial of service on layer 3 or 4 by bringing a network or service down by flooding it with large amounts of traffic	DDoS mitigation, anycast
NXDomain attack	Attacks that flood DNS server with requests for non-existent domains, causing it to send NXDomain (non-existent domain) responses	Rate limiting, caching negative answers
Phantom domain attack	Attacks where a DNS resolver is forced to resolve multiple non-existent domains, causing it to consume resources while waiting for responses	Limits on recursive queries per client and per domain

Types of DNS attacks: DNS-specific

Attack	What it does	Main defense
DNS-based exploits	Attacks that exploit bugs or vulnerabilities in the DNS software (e.g. KeyTrap, a DNSSEC validation flaw disclosed in 2024)	Patching
DNS cache poisoning	Corruption of DNS server cache data with a rogue domain or IP (e.g. the Kaminsky attack, next slide)	Source port and ID randomisation, DNSSEC
Protocol anomalies	Causing the server to crash by sending malformed DNS packets and queries	Patching, DNS-aware filtering
Reconnaissance	Attempts by hackers to get information on the network environment before launching a DDoS or other type of attack via DNS queries or Zone Transfers	Restrict zone transfers
DNS tunneling	Tunneling of another protocol through DNS port 53 for malware insertion and/or data exfiltration	DNS firewall, heuristics
DNS hijacking	Modifying the DNS record settings to point to a rogue DNS server or domain	Registrar lock, strong accounts, MFA

Cache poisoning: winning the race

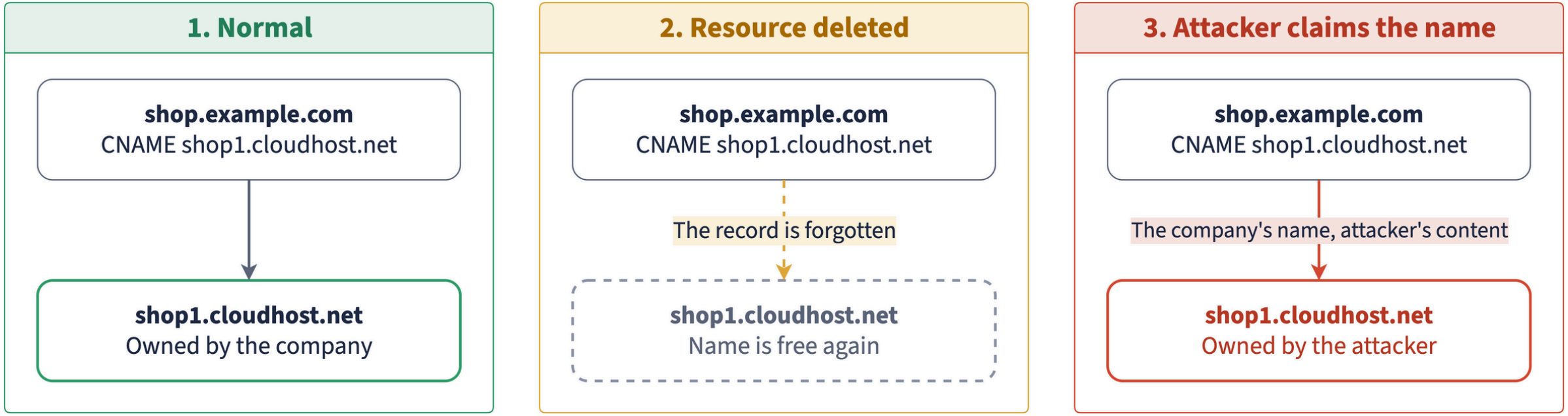


The race: the resolver accepts the first answer that matches its query. The query ID is 16 bits, so a forged answer has 1 chance in 65,536.

Kaminsky (2008): ask for random names (x1.bank.com, x2.bank.com…). None is cached, so each one is a new race with no TTL to wait for. The forged answer also carries an NS record, so the whole domain is hijacked, not one name.

Defenses: a random source port as well as a random ID (about 1 in 4 billion), then **DNSSEC**: a forged answer has no valid signature.

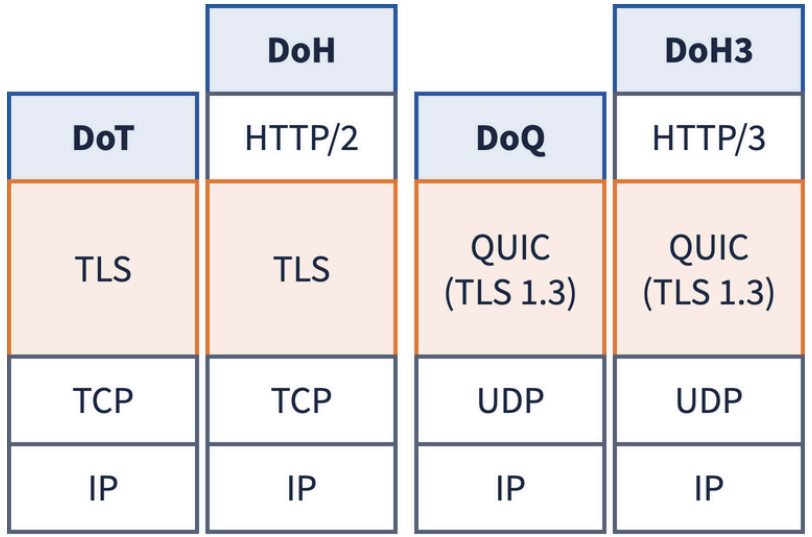
Dangling records: subdomain takeover



- **Why it works:** the record still points to a name the company no longer controls, and cloud services often let anyone claim a released name.
- **Defense:** delete the DNS record before the resource, and regularly check that every record still points to something you own.
- **Sitting Ducks**, a close relative: a domain is delegated to a DNS provider where anyone can claim it, with no access to the owner's account.

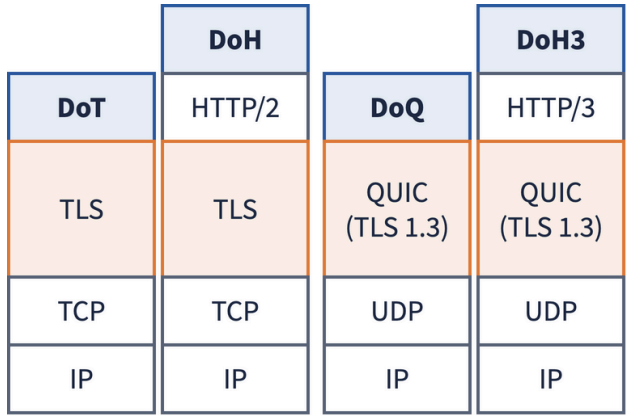
Improvements of DNS

- **DNS Transport** mechanisms
  - DNS over TLS (DoT): use DNS over TLS encryption on TCP port 853 (RFC 7858)
  - DNS over HTTPS (DoH/DoH3): use HTTP2 or 3 for encrypting requests (RFC 8484)
  - DNS-over-QUIC (DoQ): use QUIC for data transport and encryption on UDP port 853 (RFC 9250)
- **DNS Data** authentication:
  - DNSSEC: use specific DNS records to ensure that the answer given is the authorized one



Comparing DoT/DoQ and DoH/DoH3

DoT/DoQ	DoH/DoH3
RFC 7858 (DoT), RFC 9250 (DoQ)	RFC 8484
New port 853 (TCP for DoT, UDP for DoQ)	Port 443 (HTTPS)
Traffic can easily be blocked and not open normally by default in enterprise	Traffic looks like every other encrypted web traffic
No API capabilities	API capabilities
Usually set up at OS or network level	Built-in browser support
Blocking the port keeps clients on the enterprise resolver	**Can bypass the enterprise resolver, and with it the RPZ you just built**



C2 over DNS over HTTPS (DoH)

The same privacy works for attackers: C2 traffic sent to a public DoH service looks like normal HTTPS, and the enterprise resolver never sees the lookup.

Domain fronting (mostly history): hide the real destination by showing one name in the visible TLS SNI, and another in the encrypted HTTP Host header. Major providers have blocked it, Google and Amazon in 2018.

```
curl 'https://8888.google/resolve?name=example.com' -H "Host: dns.google"
```

- Unencrypted DNS lookup & TLS SNI: **8888.google**
- Encrypted HTTP host: **dns.google**
- Requested domain: **example.com** (an attacker would ask for their C2 domain)

This command works because both names belong to the same service. Tested in Nov 2026, the same trick across different services fails: Google returns 404 and Cloudflare 403.

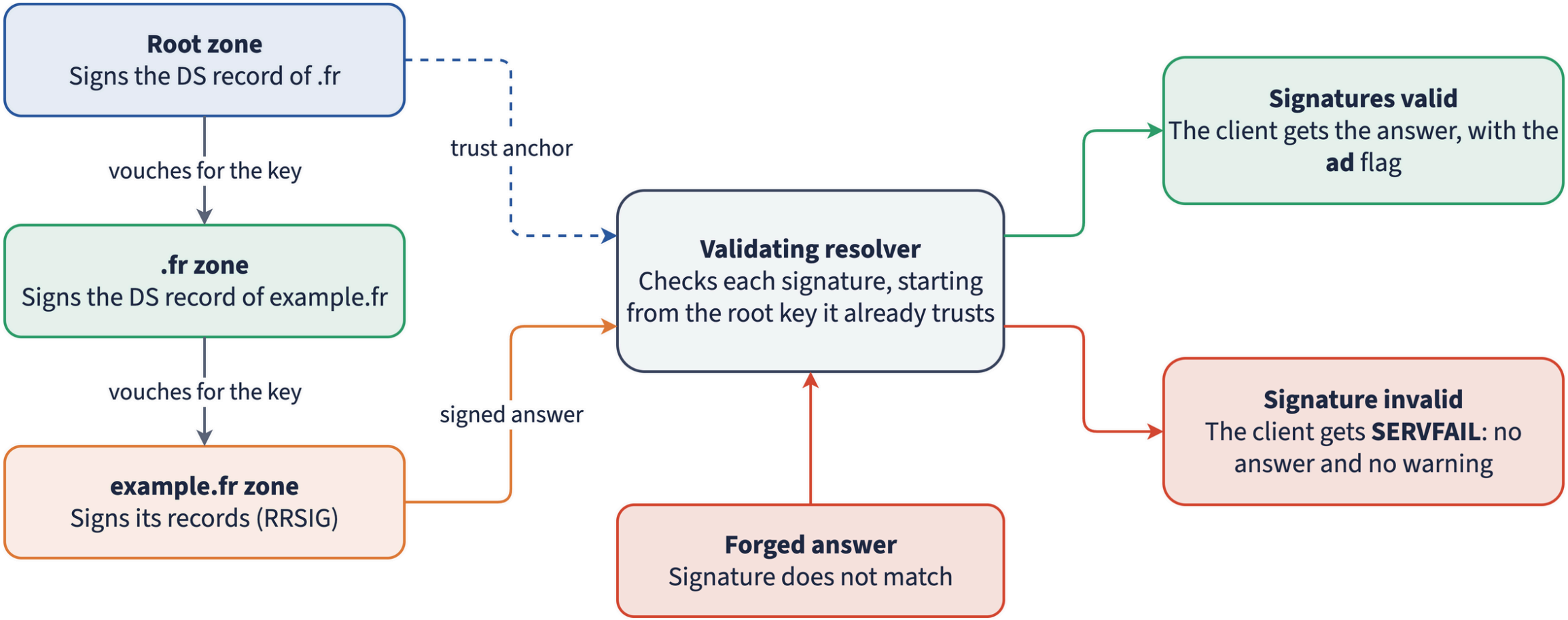
ECH (Encrypted Client Hello, RFC 9849) goes further: it hides the SNI using keys published in the site's DNS HTTPS record, so defenders lose that view too.

DNSSEC

DNS was designed without security: a resolver cannot tell a forged answer from a real one. That is what **cache poisoning** exploits.

SIGNED	Each record set carries a signature (RRSIG). A validating resolver checks it: the data is what the zone owner published, unchanged.
NOT ENCRYPTED	Answers are authenticated, not hidden: anyone on the path still reads them. DoT, DoH and DoQ do the encryption.
CHAIN OF TRUST	It follows the tree you know: the root vouches for the key of .fr (DS), .fr for the domain's key (DNSKEY), and the domain signs its records.

DNSSEC



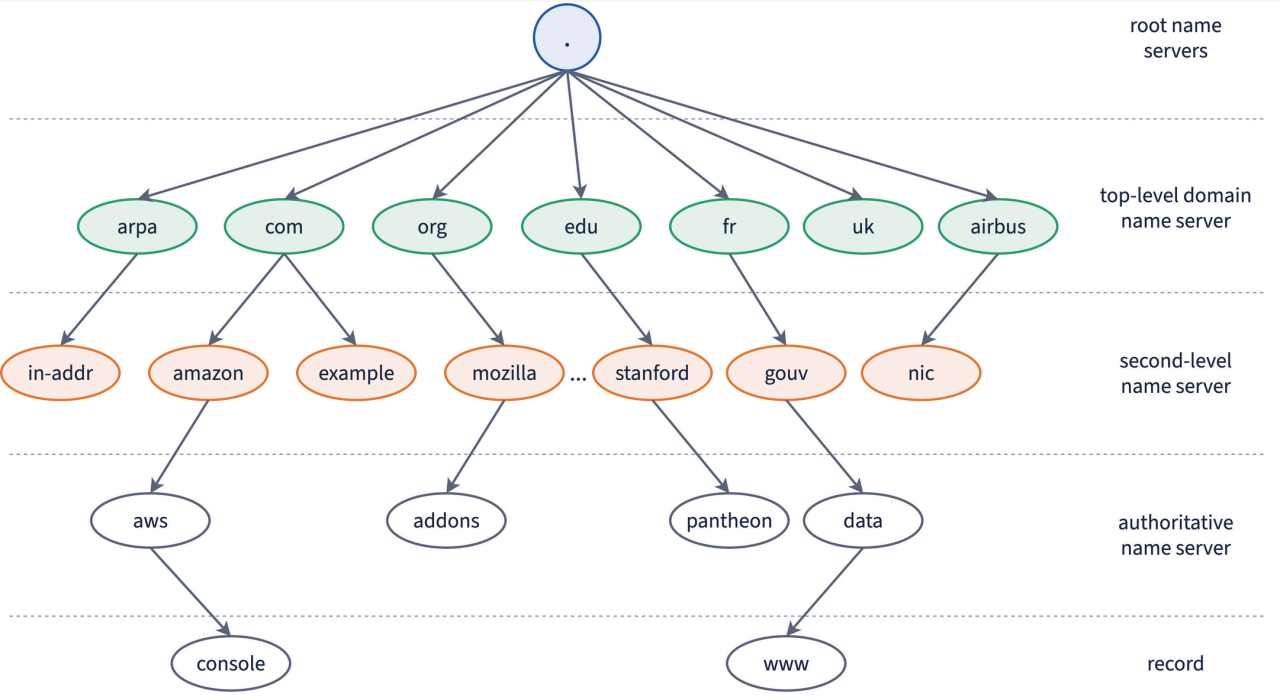
October 2026: the root key changed. A validating resolver that had not added the new key can no longer validate anything, so its lookups fail.

Agenda

1 **DNS Overview**
What is the Domain Name System?

2 **DNS Security**
How attackers abuse DNS, and how to defend it

3 **Wrap-up**
What to remember



PART 3

Wrap-up

What to remember

EXERCISE

True or false?

DNS only uses UDP.	**FALSE**	Large answers (`tc` flag) and zone transfers use TCP. DoT and DoH run over TCP too.
A DNS change “propagates” across the Internet.	**FALSE**	Nothing is pushed. Each cache keeps the old answer until its TTL expires.
Lowering a TTL takes effect at once.	**FALSE**	Caches still hold the old, long TTL. Lower it at least one old TTL before the change.
dig and the browser always see the same answer.	**FALSE**	The browser has its own cache, may use its own DoH resolver, and the hosts file wins over both.
“This name does not exist” is never cached.	**FALSE**	Negative answers are cached too, for the time set by the zone’s SOA.
DNSSEC encrypts DNS.	**FALSE**	It signs: anyone can still read the answers. DoT, DoH and DoQ encrypt.

What to remember

1. DNS is a distributed tree: zones split it up, and delegation connects them.
2. Your recursive resolver walks that tree for you: root, TLD, then authoritative server.
3. Caches keep answers until the TTL expires (or longer if the source is unreachable), so clients can see different answers for a while.
4. The same name can give different answers by design: split horizon, views, round robin.
5. DNS helps attackers at most stages of an attack, because it is allowed almost everywhere.
6. DNS can carry C2. No single indicator finds it: combine them, then block with RPZ.
7. DoT, DoH and DoQ protect the client-to-resolver path, DNSSEC proves authenticity. Neither does both, and DoH can also hide attackers.

QUIZ

Exit ticket

Before you leave · four quick questions



1. Open Dig Quest on your phone: **dns.jmanteau.fr**, same session as the dig lab. Not in yet? Scan the code on the board.
2. Each question appears on your screen: answer before the timer ends.
3. Right and fast answers score points on the **Quiz** leaderboard.

Resources

dig cheat sheet

DNS RFCs

An Illustrated Guide to the Kaminsky DNS Vulnerability

DNS for Rocket Scientists

NIST SP 800-81r3: Secure DNS Deployment Guide (2026)

MITRE ATT&CK: C2 over DNS (T1071.004)

dig cheat sheet




```
❯ dig -x 8.8.8.8 +short
dns.google.

❯ dig txt dns.google. +short
"https://xkcd.com/1361/"
"v=spf1 -all"
```